RKNN2 算子支持与约束参考

文件标识：RK-YH-YF-418

发布版本：v2.5.0

日期：2026-09-29

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

RKNN2 算子支持与约束参考

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
早期版本	NPU 团队	—	详情页
V2.0.0-beta	NPU 团队	2024-03-22	详情页
V2.1.0	NPU 团队	2024-08-07	详情页
V2.2.0	NPU 团队	2024-09-04	详情页
V2.3.0	NPU 团队	2024-11-04	详情页
V2.3.2	NPU 团队	2025-03-28	详情页
V2.4.0	NPU 团队	2026-01-15	详情页
V2.5.0	NPU 团队	2026-09-29	详情页

目录

- RKNN2 算子支持与约束参考
  - 更新历史
    - V2.5.0
    - V2.4.0
    - V2.3.2
    - V2.3.0
    - V2.2.0
    - V2.1.0
    - V2.0.0-beta
  - NPU OPs 规格介绍
    - RK3566/RK3568
      - Add
      - AddRelu
      - BatchNormalization
      - Concat
      - Convolution
      - Depthwise Convolution
      - ConvolutionRelu
      - ConvolutionClip
      - ConvolutionPRelu/LeakyRelu
      - ConvolutionAdd
      - ConvolutionSigmoid
      - ConvolutionTanh
      - ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvolutionAddRelu

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

ConvTranposeAdd

ConvTranposeSigmoid

ConvTranposeTanh

ConvTranposeSoftplus

ConvTranposeHardSigmoid

ConvTranposeHardSwish

ConvTranposeElu

ConvTranposeSwish

ConvTranposeMish

Div

Expand

GRU

Gather

LSTM

Max

Min

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Pow

Relu

Clip/ReLU6

PRelu

- LeakyRelu
- Reshape
- Resize
- Slice
- Split
- Sub
- Tile
- Transpose
- Sigmoid
- Tanh
- Softplus
- HardSigmoid
- HardSwish
- Elu
- exSwish
- exMish
- exGelu
- exSwooshR
- exSwooshL
- Where
- exGlu
- exMatMul
- exNorm
- exWindow

RK3588

- Add
- AddRelu
- BatchNormalization
- Concat
- Convolution
- Depthwise Convolution
- ConvolutionRelu
- ConvolutionClip
- ConvolutionPRelu/LeakyRelu
- ConvolutionAdd
- ConvolutionSigmoid
- ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvolutionAddRelu

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

ConvTranposeAdd

ConvTranposeSigmoid

ConvTranposeTanh

ConvTranposeSoftplus

ConvTranposeHardSigmoid

ConvTranposeHardSwish

ConvTranposeElu

ConvTranposeSwish

ConvTranposeMish

Div

Expand

GRU

Gather

Hardmax

LSTM

Max

Min

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Pow

Relu

- Clip/ReLU6
- PRelu
- LeakyRelu
- Reshape
- Resize
- Slice
- Softmax
- exSoftmax13
- exSoftmaxMask
- Split
- Sub
- Tile
- Transpose
- Sigmoid
- Tanh
- Softplus
- HardSigmoid
- HardSwish
- Elu
- exSwish
- exMish
- exGelu
- exSwooshR
- exSwooshL
- Where
- exConvStreaming
- exConvStreamingPRelu/LeakyRelu
- exGlu
- exMatMul
- exNorm
- exSDPAttention
- exWindow

RK3562

- Add
- AddRelu
- BatchNormalization
- Concat

Convolution

Depthwise Convolution

ConvolutionRelu

ConvolutionClip

ConvolutionPRelu/LeakyRelu

ConvolutionAdd

ConvolutionSigmoid

ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvolutionAddRelu

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

ConvTranposeAdd

ConvTranposeSigmoid

ConvTranposeTanh

ConvTranposeSoftplus

ConvTranposeHardSigmoid

ConvTranposeHardSwish

ConvTranposeElu

ConvTranposeSwish

ConvTranposeMish

Div

Expand

GRU

Gather

Hardmax

LSTM

Max

Mul

MulRelu

- Pad
- MaxPool
- GlobalMaxPool
- AveragePool
- GlobalAveragePool
- Pow
- Relu
- Clip/ReLU6
- PRelu
- LeakyRelu
- Reshape
- Resize
- Slice
- Softmax
- exSoftmax13
- exSoftmaxMask
- Split
- Sub
- Tile
- Transpose
- Sigmoid
- Tanh
- Softplus
- HardSigmoid
- HardSwish
- Elu
- exSwish
- exMish
- exGelu
- Where
- exGlu
- exMatMul
- exNorm
- exSDPAttention
- exWindow

RV1103/RV1106

- Add

AddRelu

BatchNormalization

Concat

Convolution

Depthwise Convolution

ConvolutionRelu

ConvolutionClip

ConvolutionPRelu/LeakyRelu

ConvolutionAdd

ConvolutionSigmoid

ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvolutionAddRelu

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

ConvTranposeAdd

ConvTranposeSigmoid

ConvTranposeTanh

ConvTranposeSoftplus

ConvTranposeHardSigmoid

ConvTranposeHardSwish

ConvTranposeElu

ConvTranposeSwish

ConvTranposeMish

Div

Expand

Hardmax

LSTM

Max

Min

- Mul
- MulRelu
- Pad
- MaxPool
- GlobalMaxPool
- AveragePool
- GlobalAveragePool
- Pow
- Relu
- Clip/ReLU6
- PRelu
- LeakyRelu
- Reshape
- Resize
- Slice
- Softmax
- exSoftmax13
- exSoftmaxMask
- Split
- Sub
- Tile
- Transpose
- Sigmoid
- Tanh
- Softplus
- HardSigmoid
- HardSwish
- Elu
- exSwish
- exMish
- exGelu
- Where
- exGlu
- exMatMul
- exSDPAttention
- exWindow

RK3576/RV1126B

Add

AddRelu

BatchNormalization

Concat

Convolution

Depthwise Convolution

ConvolutionRelu

ConvolutionClip

ConvolutionPRelu/LeakyRelu

ConvolutionAdd

ConvolutionSigmoid

ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvolutionAddRelu

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

ConvTranposeAdd

ConvTranposeSigmoid

ConvTranposeTanh

ConvTranposeSoftplus

ConvTranposeHardSigmoid

ConvTranposeHardSwish

ConvTranposeElu

ConvTranposeSwish

ConvTranposeMish

Div

Expand

GRU

Gather

Hardmax

LSTM

Max

Greater

Min

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Pow

ReduceMax

Relu

Clip/ReLU6

PRelu

LeakyRelu

Reshape

Resize

Slice

Softmax

exSoftmax13

exSoftmaxMask

Split

Sub

Tile

Transpose

Sigmoid

Tanh

Softplus

HardSigmoid

HardSwish

Elu

exSwish

exMish

exGelu

Sin

- Cos
- Where
- exConvStreaming
- exConvStreamingPRelu/LeakyRelu
- exGlu
- exMatMul
- exNorm
- exSDPAttention
- exWindow

RK2118

- Add
- AddRelu
- BatchNormalization
- Concat
- Convolution
- Depthwise Convolution
- ConvolutionRelu
- ConvolutionClip
- ConvolutionPRelu/LeakyRelu
- ConvolutionAdd
- ConvolutionSigmoid
- ConvolutionTanh
- ConvolutionSoftplus
- ConvolutionHardSigmoid
- ConvolutionHardSwish
- ConvolutionElu
- ConvolutionSwish
- ConvolutionMish
- ConvTranspose
- Div
- Expand
- LSTM
- Max
- Mul
- MulRelu
- Pad
- MaxPool

- GlobalMaxPool
- AveragePool
- GlobalAveragePool
- Pow
- Relu
- Clip/ReLU6
- PRelu
- LeakyRelu
- Reshape
- Resize
- Slice
- Softmax
- exSoftmax13
- exSoftmaxMask
- Split
- Sub
- Tile
- Transpose
- Sigmoid
- Tanh
- Softplus
- HardSigmoid
- HardSwish
- Elu
- exSwish
- exMish
- exGelu
- Where
- exConvStreaming
- exConvStreamingPRelu/LeakyRelu
- exGlu
- exMatMul
- exWindow

RV1103B/RV1106B

- Add
- AddRelu
- BatchNormalization

Concat

Convolution

Depthwise Convolution

ConvolutionRelu

ConvolutionClip

ConvolutionPRelu/LeakyRelu

ConvolutionAdd

ConvolutionSigmoid

ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

Div

Expand

Max

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Relu

Clip/ReLU6

PRelu

LeakyRelu

Reshape

Resize

Slice

Softmax

exSoftmax13
exSoftmaxMask
Split
Sub
Tile
Transpose
Sigmoid
Tanh
Softplus
HardSigmoid
HardSwish
Elu
exSwish
exMish
exGelu
Where
exGlu
exMatMul
exWindow
CPU OPs 规格介绍
注:
Add
AveragePool
ArgMin
ArgMax
BatchNormalization
Cast
Clip
Concat
Convolution
ConvTranspose/Deconvolution
ConvTransposePad
Cos
DataConvert
DepthToSpace
Div

Equal

Exp

Flatten

Gather

Greater

GreaterOrEqual

GRU

exGRU

exHardSwish

InstanceNormalization

exLayerNorm

Less

LessOrEqual

LogSoftmax

LpNormalization

exLRN

MatMul

Max

MaxPool

MaxRoiPool

MaxUnpool

exMish

Min

Mul

Pad

Pow

exProposal

ReduceMax

ReduceMean

ReduceSum

ReduceMin

Reorg

Reshape

Resize

ReverseSequence

exRMSNorm

RoiAlign

ScatterND

Sin

Slice

Softmax

exSoftmax13

SpaceToDetph

Split

Sqrt

Squeeze

Sub

Tanh

Tile

Transpose

Upsample

Not

where

Erf

Floor

Mod

exMeanVarianceNormalization

And

GatherElements

Log

exDataConvert

SpaceToDepth

SoftmaxMask

TransposeReshape

ReshapeTranspose

ReduceL2

LayerNormalization

PassThrough

Expand

Sigmoid

Tanh

Cos

Sin

exSwish

exGlu
exGelu
exWindow
exNorm
Round
TopK
exSwooshR
exSwooshL
OneHot
Cumsum
GPU OPs 规格介绍
注:
GridSample
GRU
exGelu
模型输入输出限制说明
模型输入限制说明
RK3566/RK3568
RK3588
RV1103/RV1106
RK3562
RK3576/RV1126B
RK2118
RV1103B
模型输出限制说明
RK3566/RK3568
RK3588
RV1103/RV1106
RK3562
RK3576/RV1126B
RK2118
RV1103B/RV1106B
特殊说明
dilated_kernel_h
dilated_kernel_w

更新历史

V2.5.0

修改人：NPU 团队

修改日期：2026-09-29

修改内容

1. 版本号更新为 V2.5.0

V2.4.0

修改人：NPU 团队

修改日期：2026-01-15

修改内容

1. 新增GPU 算子支持文档
2. CPU 算子支持列表新增Pow/CumSum
3. 3588NPU 实现新增ReduceMax+exConvStreaming算子文档
4. 3576NPU 实现新增ReduceMax/exConvStreaming/Sin/Cos算子文档
5. 1126BNPU 实现新增ReduceMax+exConvStreaming算子文档

V2.3.2

修改人：NPU 团队

修改日期：2025-03-28

修改内容

1. 增加RV1126B平台文档
2. 增加RV1106B平台文档
3. 更新全平台ConvDilation参数限制
4. 更新Resize算子限制

V2.3.0

修改人：NPU 团队

修改日期：2024-11-04

修改内容

1. 增加RK3576/RK3588/RK3562/RK3568平台exNorm的mean_var模式文档说明
2. 全平台MAX/MINOP支持Broadcast
3. 修改RK3576Softmax限制规格

V2.2.0

修改人：NPU 团队

修改日期：2024-09-03

修改内容

1. 全平台(除rk3568)修改softmax规格限制,新增softmax切分
2. RK3576、RK3588、RK3562、RK3568新增RMSNorm支持

V2.1.0

修改人：NPU 团队

修改日期：2024-08-07

修改内容

1. 新增RK2118文档说明
2. 新增RV1103B文档说明
3. 全平台算子规格更新
4. RK3576算子规格新增(包含Hardmax、ConvLut等)
5. RK2118exConvStreaming支持
6. RK3562/RK3576exSDPA算子添加FlashAttentionV2优化支持

V2.0.0-beta

修改人：NPU 团队

修改日期：2024-03-22

修改内容

1. 新增RK3576平台算子支持规格
2. 新增RK3588/RK3562/RK3576平台的exSDPAttention、exMatMul的硬件支持规格
3. 新增Floor、Mod、And、GatherElements以及exMeanVarianceNormalizationCPU 算子支持
4. 新增RK2118平台算子支持规格

NPU OPs 规格介绍

RK3566/RK3568

Add

执行元素级二进制加法(C=A+B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

AddRelu

Add与Relu融合计算,C=Relu(A+B)

输入列表

- 同Add

输出列表

- 同Add

其他支持

- 多核联合:支持

BatchNormalization

按照论文https://arxiv.org/abs/1502.03167中的描述对输入张量进行批量归一化计算

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- epsilon:float（默认值=1e-5）
    - 除以标准差时加上防止除0的实数
    - epsilon∈(0,∞]
- momentum:float
    - 训练时的滑动平均参数

Concat

将一系列向量在axis指定的方向上组合成一个向量，所有向量除axis指定的方向外大小必须相同

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axisint64
  - axis∈{0,1,2,3}

Convolution

卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持

        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Depthwise Convolution

深度可分离卷积

输入列表

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
    - 量化支持
        - per-layer

- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

ConvolutionRelu

Convolution与Relu融合计算,Output=Relu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionClip

Convolution与Clip融合计算,Output=Clip(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution 与 PRelu/LeakyRelu 融合计算, Output = PRelu(Conv(Input)) 或者 Output = LeakyRelu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionAdd

Convolution与Add融合计算,Output=Add(Conv(Input0),Input1) 说明及规格限制同Convolution

---

ConvolutionSigmoid

Convolution与Sigmoid融合计算,Output=Sigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionTanh

Convolution与Tanh融合计算,Output=Tanh(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionSoftplus

Convolution与Softplus融合计算,Output=Softplus(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSigmoid

Convolution与HardSigmoid融合计算,Output=HardSigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSwish

Convolution与HardSwish融合计算,Output=HardSwish(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionElu

Convolution与Elu融合计算,Output=Elu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionSwish

Convolution与Swish融合计算,Output=Swish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionMish

Convolution与Mish融合计算,Output=Mish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionAddRelu

Convolution与Add及Relu融合计算,Output=Relu(Add(Conv(Input0),Input1)) 说明及规格限制同Convolution

ConvTranspose

转置卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
- 量化支持
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)

    - stride_h仅支持1,2,4,8
    - stride_w仅支持1,2,4,8
- pads:int(pads_top,pads_left,pads_bottom,pads_right) 设置pad时注意: 不支持kernel_h*dilations_h-dilations_h-pads_top<0 不支持kernel_w*dilations_w-dilations_w-pads_left<0 不支持stride_h*(height-1)-pads_top+1<output_h 不支持stride_w*(width-1)-pads_left+1<output_w
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group:仅支持1,当且仅当num_input=num_output时支持num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - 当dilation不等于(1,1)时，只支持kernel=(1,1)

数据类型约束

- T1:int8(Tensor),float16(Tensor)

ConvTranposeRelu

ConvTranpose与Relu融合计算,Output=Relu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeClip

ConvTranpose与Clip融合计算,Output=Clip(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranpose与PRelu/LeakyRelu融合计算,Output=PRelu(ConvTranpose(Input))或者Output=LeakyRelu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeAdd

ConvTranpose与Add融合计算,Output=Add(ConvTranpose(Input0),Input1) 说明及规格限制同ConvTranspose

---

ConvTranposeSigmoid

ConvTranpose与Sigmoid融合计算,Output=Sigmoid(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeTanh

ConvTranpose与Tanh融合计算,Output=Tanh(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSoftplus

ConvTranpose与Softplus融合计算,Output=Softplus(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranpose与HardSigmoid融合计算,Output=HardSigmoid(ConvTranpose(Input))

ConvTranposeHardSwish

ConvTranpose与HardSwish融合计算,Output=HardSwish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeElu

ConvTranpose与Elu融合计算,Output=Elu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSwish

ConvTranpose与Swish融合计算,Output=Swish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeMish

ConvTranpose与Mish融合计算,Output=Mish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

Div

执行元素级二进制除法(C=A/B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

根据给定的 shape 和广播规则广播输入张量。广播规则类似于 numpy.array(input) * numpy.ones(shape)：维度右对齐；两个对应维度必须具有相同的值，或者其中一个等于1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

- shape:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

广播约束

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

GRU

门控循环单元（GRU，GatedRecurrentUnit） GRU扩展以及变体命名为exGRU算子，参数项中指明（extern）的项为exGRU独有的参数项。 计算公式如下： Equations(Default:f=Sigmoid,g=Tanh): r_t = f(x_t · W_r + h_{t−1} · R_r + Wb_r + Rb_r) z_t = f(x_t · W_z + h_{t−1} · R_z + Wb_z + Rb_z) h_t = g(x_t · W_h + (r_t ⊙ h_{t−1}) · R_h + Rb_h + Wb_h), (WRB=0) h_t = g(x_t · W_h + r_t ⊙ (h_{t−1} · R_h + Rb_h)) + Wb_h, (WRB=1) H_t = (1 − z_t) ⊙ h_t + z_t ⊙ h_{t−1}

输入列表

- X:T1
    - shape:[seq_length,batch_size,input_size]
        - seqlength∈(0,8192]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - inputsize∈(0,8192]
    - 量化支持
        - per-layer
- W:T1
    - shape:[num_directions,4*hidden_size,input_size]

        - inputsize∈(0,8192]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- R:T1
    - shape:[num_directions,4*hidden_size,hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- B:T2
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- sequence_lens:T3(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]

        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

属性列表

- linear_before_reset:int
    - LBR变种的选择:1(T)or0(F)
- direction(extern):string
    - 指定GRU的运算方向
    - forward:指定GRU的运算方向为前向
    - reverse:指定GRU的运算方向为反向
    - bidirectional:指定GRU的运算方向为双向
- sequence_size(extern):int
    - 指定GRU输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
    - GRU单元中的hiddensize，无限制，建议8对齐
- input_layout(extern):int
    - 指定与对应输入shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
    - snc：指定layout对应的输入shape为[seqs,batches,input_size]
    - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
    - 指定与对应输出shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。

    - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
    - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

其他支持

- 多核联合:暂不支持

Gather

根据索引Indices获取输入X的指定axis维度的条目，并将它们拼接在一起。

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- Indices:T2
    - 要收集的元素的索引，秩rank =1

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int(Tensor)

属性列表

- axis:int(Tensor)
    - axis∈{0,1,2,3}
    - 指定Indices获取输入的维度

其他支持

- 多核联合:尚不支持

LSTM

长短期记忆网络（LSTM，Long Short-Term Memory） LSTM扩展以及变体命名为exLSTM算子，参数项中指明 （extern）的项为exLSTM独有的参数项。 计算公式如下：
Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

输入列表

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - inputsize∈(0,8192]
  - 量化支持
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- B:T2,T3
  - shape:[num_directions,8*hidden_size]

        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- sequence_lens:T4,T5(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]

        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

属性列表

- direction(extern):string
    - 指定LSTM的运算方向
    - forward:指定LSTM的运算方向为前向
    - reverse:指定LSTM的运算方向为反向
    - bidirectional:指定LSTM的运算方向为双向
- sequence_size(extern):int
    - 指定LSTM输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
    - LSTM单元中的hiddensize，无限制，建议8对齐
- proj_size(extern):int
    - projection时的proj_size，projsize∈[0,hiddensize]
    - 目前限定0，即尚不支持projection功能
- input_forget(extern):int
    - cifg变种的选择:1(T)or0(F)目前限定0，即尚不支持
- has_projection(extern):int
    - projection变种:1(T)or0(F)目前限定0，即尚不支持
- input_layout(extern):int
    - 指定与对应输入shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
    - snc：指定layout对应的输入shape为[seqs,batches,input_size]

    - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
    - 指定与对应输出shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
    - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
    - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int32(Tensor)
- T4:float(Scalar)
- T5:int(Scalar)

其他支持

- 多核联合:暂不支持

Max

计算输入张量的元素级最大值

输入列表

- A:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Min

计算输入张量的元素级最小值

输入列表

- A:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Mul

执行元素级二进制乘法(C=A*B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

- T2:float(constTensor)

MulRelu

其余说明同Mul

- 支持的数据类型: 同Mul

Pad

给定一个包含要填充的数据的张量(data),包含轴的开始和结束填充数值的张量(pads),模式(mode),常 量数值(constant_value),生成一个填充张量(output)

输入列表

- data:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

属性列表

- mode:string(默认值constant)
  - 支持模式:constant,reflect
    - constant无限制
    - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

其他支持

- 多核联合:尚不支持

MaxPool

MaxPool消耗输入张量X并根据内核大小、步幅大小和pad长度在张量上应用最大池化。最大池化包括 根据内核大小计算输入张量子集的所有值的最大值，并将数据下采样到输出张量Y中以进行进一步处理。 输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使用自 动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]

- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]
- storage_order:int64
  - 优先储存方式:0

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool使用输入张量X并对同一通道中的值应用最大池化。这相当于MaxPool的kernel_shape 大小等于input的空间维度。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(1,7]
    - width∈(1,7]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET

- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool使用输入张量X并根据内核大小、步幅大小和pad长度在张量上应用平均池化。平均池化 包括根据内核大小计算输入张量子集的所有值的平均值，并将数据下采样到输出张量Y中以进行进一步 处理。输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使 用自动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)

    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool使用输入张量X并对同一通道中的值应用平均池化。这相当于AveragePool，其kernel_shape大小等于输入张量的空间维度。

输入列表*

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z =XY 指数运算，采用输入数据（张量）和指数，并产生一个输出数据（张量）

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- Y:T2
  - shape:[1]
    - 当前仅支持0、1、2、3、0.5、-0.5

输出列表

- Z:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

其他支持

- 多核联合:尚不支持

Relu

将输入张量的负值设为零，正值保持不变

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 无

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Clip/ReLU6

将输入张量的值裁剪到[0,6]范围内

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 无

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

PRelu

将输入张量的负值按可学习参数缩放，正值保持不变

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer/per-channel
  - 广播支持
    - 无

输出列表

- output:T1

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

属性列表

- slope:PRelu系数
    - 仅支持单个标量或C维度系数

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

LeakyRelu

将输入张量的负值按固定比例缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- shape:int
    - 指定输出张量的形状

其他支持

- 多核联合:尚不支持

Resize

调整输入张量的大小。一般来说，它将输出张量中的每个值计算为输入张量中邻域（即采样位置）的加 权平均值

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- roi(optional):T2
    - 目前暂不支持

- scales(optional):T2
  - 沿每个维度的缩放比例数组
- sizes(optional):T3
  - 输出张量的目标大小

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

属性列表

- antialias:int（默认值为0）
  - 目前不支持设置
- axes:int[]
  - 目前不支持设置
- coordinate_transformation_mode:strings（默认值为”half_pixel”）
  - 目前仅支持half_pixel，pytorch_half_pixel，align_corners三种
- cubic_coeff_a:float（默认为-0.75）
  - 目前不支持设置
- exclude_outside:int（默认值为0）
  - 目前不支持设置
- extrapolation_value:float（默认为0）
  - 目前不支持设置
- keep_aspect_ratio_policy:strings（默认值为”stretch”）
  - 目前不支持设置
- mode:strings（默认值为”nearest”）
  - 目前仅支持nearest和linear两种可配置
- nearest_mode:strings（默认值为”round_prefer_floor”）
  - 目前不支持设置 支持说明 目前仅支持宽高方向的整数倍缩放,且缩放倍数为2-8的整数倍,如果是(1,c,1,1)则支持放大到(1,c,h, w)

Slice

获得当前向量的一个切片，具体规则和numpy切片相同

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- starts:int(Tensor)
  - 切分的起始位置:无限制
- ends:int(Tensor)
  - 切分的终止位置:无限制
- axes:int(Tensor)
  - 选取切分的轴:支持任意0~3轴，支持同时多轴选择
- steps:int(Tensor)
  - 选取切分对应轴的步长
  - 当height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]%subc==0&&steps[1]<=subc时,steps可以不为1，否则只有steps==[1,1,1,1]时，才可 通过NPU运行SliceOP

Split

将向量沿着axis方向分成num_outputs个向量，split用于指定切分后向量在axis方向上的大小

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - 一或多个输出向量，由num_outputs属性决定
  - shape:根据切分情况决定
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- 执行元素级二进制减法(C=A-B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer
    - 广播支持
        - [b,c,1,1]
        - [scalar]

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Tile

通过平铺给定的张量构造张量,这与Numpy中的函数tile相同，但没有广播

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

对输入张量进行转置

输入列表

- X:T1
  - shape:[n1,c1,h1,w1]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

输出列表

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

属性列表

- perm转置的轴顺序: 不同perm参数限制如下：(其中p=8192,q=2048)
  - perm=[0,1,2,3]
    - 无限制
  - perm=[0,1,3,2]
    - 不支持
  - perm=[0,2,3,1]
    - 不支持
  - perm=[0,2,1,3]
    - 不支持
  - perm=[0,3,1,2]
    - 不支持
  - perm=[0,3,2,1]
    - 不支持
  - perm=[1,0,2,3]
    - 不支持
  - perm=[1,0,3,2]

    - 不支持
- perm=[1,2,0,3]
    - 不支持
- perm=[1,2,3,0]
    - 不支持
- perm=[1,3,0,2]
    - 不支持
- perm=[1,3,2,0]
    - 不支持
- perm=[2,1,0,3]
    - T1,T2:c1<8192,n<8192,h1*w1<8192,(h1=w1=1,n1%4=0)or(n1=h1=1,h1%4=0)
- perm=[2,1,3,0]
    - T1,T2:c1<8192,n<8192,h1*w1<8192,(h1=w1=1,n1%4=0)or(n1=h1=1,h1%4=0)
- perm=[2,0,3,1]
    - 不支持
- perm=[2,0,1,3]
    - 不支持
- perm=[2,3,1,0]
    - 不支持
- perm=[2,3,0,1]
    - 不支持
- perm=[3,0,2,1]
    - 不支持
- perm=[3,0,1,2]
    - 不支持
- perm=[3,2,0,1]
    - 不支持
- perm=[3,2,1,0]
    - 不支持
- perm=[3,1,0,2]
    - T1,T2:c1<8192,n<8192,h1*w1<8192,(h1=w1=1,n1%4=0)or(n1=h1=1,w1%4=0)
- perm=[3,1,2,0]
    - T1,T2:c1<8192,n<8192,h1*w1<8192,(h1=w1=1,n1%4=0)or(n1=h1=1,w1%4=0)

数据类型约束

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Sigmoid

给定输入张量，函数y=1/(1+exp(-x))按元素应用于输入张量，得到输出张量

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Tanh

计算给定输入张量单元的双曲正切

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Softplus

按元素应用Softplus函数,y=ln(exp(x)+1)

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

HardSigmoid

给定输入张量，函数y=max(0,min(1,alpha*x+beta))按元素应用于输入张量，得到输出张量，这里alpha=1/6，beta=0.5

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

HardSwish

按元素应用 HardSwish 函数, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x),这里alpha=1/6,beta=0.5

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Elu

给定输入张量，函数f(x)=alpha*(exp(x)-1.)forx<0,f(x)=xforx>=0按元素应用于输入张量，得到 输出张量，这里alpha=1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exSwish

按元素应用Sigmoid线性单元函数,Swish函数也称为SiLU函数,y=x*sigmoid(x)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exMish

给定输入张量，函数mish(x)=x*tanh(softplus(x))*=xtanh(ln(1+e^{x}))按元素应用于输入张量，得 到输出张量

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

exGelu

给定输入张量，函数y =qx∗Φ(x)按元素应用于输入张量，得到输出张量，当Φ(x)函数设置成tanh时， y =0.5∗x∗(1+Tanh(2 ∗(x+0.044715∗x3))) π

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exSwooshR

给定输入张量，按论文https://arxiv.org/abs/2310.11230给定的下列函数，按元素应用于输入张量，得 到输出张量 SwooshR(x)=log(1+exp(x−1))−0.08x−0.313261687

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exSwooshL

给定输入张量，按论文https://arxiv.org/abs/2310.11230给定的下列函数，按元素应用于输入张量，得 到输出张量 SwooshL(x)=log(1+exp(x−4))−0.08x−0.035

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Where

根据mask_tensor获取x_tensor或y_tensor的值，当mask_tensor位置为True时，取x_tensor对 应位置的值，否则取y_tensor的值。

输入列表

- mask_tensor:T1
  - shape:[batch,channel,height,width]
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束
    - 广播约束同Expand算子
- x_tensor:*T2
  - shape:[batch,channel,height,width]

    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子
- y_tensor:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子

输出列表

- output:T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exGlu

门控线性单元(GatedLinearUnit)函数,其中a是输入矩阵的前一半,b是后一半 GLU(a,b)=a⊗σ(b)

输入列表

- Input:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Output:T1

    - shape:[batch,channel/2,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int(默认值=1)
    - 分割输入的维度
    - 约束:仅支持1

---

exMatMul

矩阵乘法操作

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

输入列表

- A:T1
    - shape:[b,k,1,n]
        - k ∈(0,8192]
    - 量化支持
        - per-layer
- B:T1
    - shape:[b,k,1,m]
    - 量化支持
        - per-layer
- C(optional):T1,T2
    - shape:[b,n,1,m]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- Y:T1
  - shape:[b,n,1,m]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)

属性列表

- c_type:strings(默认值“add”)
  - 支持add和mul可选,表示C是做加法还是乘法

---

exNorm

对输入数据进行各种归一化操作，目前支持LayerNormalization、RMSNormalization和MeanVarianceNormalization三种：

$$\mathrm{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot W + B$$

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{RMS}^2 + \epsilon}} \cdot W + B$$

$$\mathrm{MeanVarianceNorm}(x) = \frac{x - \mu}{\sqrt{\mathrm{Variance} + \epsilon}} \cdot W + B$$

其中： x是输入向量。 μ和σ²分别是输入向量的均值和方差：

$$\mu = \frac{1}{H} \sum_{i=1}^{H} x_i$$

$$\sigma^2 = \frac{1}{H} \sum_{i=1}^{H} (x_i - \mu)^2$$

RMS是输入向量均方根：

$$\mathrm{RMS} = \sqrt{\frac{1}{H} \sum_{i=1}^{H} x_i^2}$$

Variance是输入向量的等效方差：

$$\text{Variance} = \frac{1}{H}\sum_{i=1}^{H} x_i^2 - \mu^2$$

ϵ是一个很小的数，用于防止除零操作。 W 和B是可选择的参数。

输入列表

- X:T1
  - shape:[batch,channel,height,width]
- W(optional):T1
  - shape:[batch,channel,height,width]
- B(optional):T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

属性列表

- epsilon:float(默认值1e-05)
  - 用来防止除0的epsilon数值
- mode:string
  - 支持layer、rms和mean_var三种Normalization模式可选
- axes:int64[](默认值[1])
  - 归一化轴，目前只支持[1]和[1,2,3]

其他支持

- 多核联合:尚不支持

---

exWindow

exWindow来源于swin_transformer的window_partition和window_reverse操作。 原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size× window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。 pytorch实现的计算方法如下：

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

输入列表

- X:T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- mode:string
  - 支持模式:partition,reverse,partition_num_first
    - partition:参考window_partition实现;
    - reverse:参考window_reverse实现;
    - partition_num_first: 与 partition 的区别在于，partition_num_first 的 H,W 按照num_windows进行划分，而partition的H,W按照window_sizes进行划分。

- window_sizes:listofints
    - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是 基于这个大小。
- num_windows:listofints
    - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。

其他支持

- 多核联合:尚不支持

RK3588

Add

执行元素级二进制加法(C=A+B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

其他支持

- 多核联合:支持

AddRelu

Add与Relu融合计算,C=Relu(A+B)

输入列表

- 同Add

输出列表

- 同Add

其他支持

- 多核联合:支持

BatchNormalization

按照论文https://arxiv.org/abs/1502.03167中的描述对输入张量进行批量归一化计算

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- epsilon:float（默认值=1e-5）
  - 除以标准差时加上防止除0的实数
  - epsilon∈(0,∞]
- momentum:float
  - 训练时的滑动平均参数

其他支持

- 多核联合:尚不支持

Concat

将一系列向量在axis指定的方向上组合成一个向量，所有向量除axis指定的方向外大小必须相同

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axisint64
  - axis∈{0,1,2,3}

Convolution

卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,31]
  - dilations_w∈[1,31]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:支持

Depthwise Convolution

深度可分离卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]

    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:支持

ConvolutionRelu

Convolution与Relu融合计算,Output=Relu(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:支持

ConvolutionClip

Convolution与Clip融合计算,Output=Clip(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:支持

ConvolutionPRelu/LeakyRelu

Convolution 与 PRelu/LeakyRelu 融合计算, Output = PRelu(Conv(Input)) 或者 Output = LeakyRelu(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:支持

ConvolutionAdd

Convolution与Add融合计算,Output=Add(Conv(Input0),Input1) 说明及规格限制同Convolution

其他支持

- 多核联合:支持

ConvolutionSigmoid

Convolution与Sigmoid融合计算,Output=Sigmoid(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:暂不支持

ConvolutionTanh

Convolution与Tanh融合计算,Output=Tanh(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:暂不支持

ConvolutionSoftplus

Convolution与Softplus融合计算,Output=Softplus(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:暂不支持

ConvolutionHardSigmoid

Convolution与HardSigmoid融合计算,Output=HardSigmoid(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:暂不支持

ConvolutionHardSwish

Convolution与HardSwish融合计算,Output=HardSwish(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:暂不支持

ConvolutionElu

Convolution与Elu融合计算,Output=Elu(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:暂不支持

ConvolutionSwish

Convolution与Swish融合计算,Output=Swish(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:暂不支持

ConvolutionMish

Convolution与Mish融合计算,Output=Mish(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:暂不支持

ConvolutionAddRelu

Convolution与Add及Relu融合计算,Output=Relu(Add(Conv(Input0),Input1)) 说明及规格限制同Convolution

其他支持

- 多核联合:支持

ConvTranspose

转置卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
- 量化支持
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]

    - 量化支持
        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h仅支持1,2,4,8
    - stride_w仅支持1,2,4,8
- pads:int(pads_top,pads_left,pads_bottom,pads_right) 设置pad时注意: 不支持 kernel_h*dilations_h-dilations_h-pads_top<0 不支持kernel_w*dilations_w-dilations_w-pads_left<0 不支持stride_h*(height-1)-pads_top+1<output_h 不支持stride_w*(width-1)-pads_left+1<output_w
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group:仅支持1,当且仅当num_input=num_output时支持num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - 当dilation不等于(1,1)时，只支持kernel=(1,1)

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

ConvTranposeRelu

ConvTranpose与Relu融合计算,Output=Relu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeClip

ConvTranpose与Clip融合计算,Output=Clip(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranpose与PRelu/LeakyRelu融合计算,Output=PRelu(ConvTranpose(Input))或者Output=LeakyRelu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeAdd

ConvTranpose与Add融合计算,Output=Add(ConvTranpose(Input0),Input1) 说明及规格限制同ConvTranspose

---

ConvTranposeSigmoid

ConvTranpose与Sigmoid融合计算,Output=Sigmoid(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeTanh

ConvTranpose与Tanh融合计算,Output=Tanh(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSoftplus

ConvTranpose与Softplus融合计算,Output=Softplus(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranpose与HardSigmoid融合计算,Output=HardSigmoid(ConvTranpose(Input))

ConvTranposeHardSwish

ConvTranpose与HardSwish融合计算,Output=HardSwish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeElu

ConvTranpose与Elu融合计算,Output=Elu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSwish

ConvTranpose与Swish融合计算,Output=Swish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeMish

ConvTranpose与Mish融合计算,Output=Mish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

Div

执行元素级二进制除法(C=A/B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

其他支持

- 多核联合:不支持

Expand

根据给定的 shape 和广播规则广播输入张量。广播规则类似于 numpy.array(input) * numpy.ones(shape)：维度右对齐；两个对应维度必须具有相同的值，或者其中一个等于1

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- shape:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

广播约束

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

其他支持

- 多核联合:不支持

GRU

门控循环单元（GRU，GatedRecurrentUnit） GRU扩展以及变体命名为exGRU算子，参数项中指明（extern）的项为exGRU独有的参数项。 计算公式如下： Equations(Default:f=Sigmoid,g=Tanh): r_t = f(x_t · W_r + h_{t−1} · R_r + Wb_r + Rb_r) z_t = f(x_t · W_z + h_{t−1} · R_z + Wb_z + Rb_z) h_t = g(x_t · W_h + (r_t ⊙ h_{t−1}) · R_h + Rb_h + Wb_h), (WRB=0) h_t = g(x_t · W_h + r_t ⊙ (h_{t−1} · R_h + Rb_h)) + Wb_h, (WRB=1) H_t = (1 − z_t) ⊙ h_t + z_t ⊙ h_{t−1}

输入列表

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - inputsize∈(0,8192]
  - 量化支持
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- B:T2
  - shape:[num_directions,8*hidden_size]

        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- sequence_lens:T3(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

属性列表

- linear_before_reset:int
  - LBR变种的选择:1(T)or0(F)
- direction(extern):string
  - 指定GRU的运算方向
  - forward:指定GRU的运算方向为前向
  - reverse:指定GRU的运算方向为反向
  - bidirectional:指定GRU的运算方向为双向
- sequence_size(extern):int
  - 指定GRU输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
  - GRU单元中的hiddensize，无限制，建议8对齐
- input_layout(extern):int
  - 指定与对应输入shape含义一致的layout
  - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
  - snc：指定layout对应的输入shape为[seqs,batches,input_size]
  - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
  - 指定与对应输出shape含义一致的layout
  - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
  - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
  - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

其他支持

- 多核联合:暂不支持

Gather

根据索引Indices获取输入X的指定axis维度的条目，并将它们拼接在一起。

输入列表

- X:T1

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- Indices:T2
    - 要收集的元素的索引，秩rank =1

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int(Tensor)

属性列表

- axis:int(Tensor)
    - axis∈{0,1,2,3}
    - 指定Indices获取输入的维度

其他支持

- 多核联合:尚不支持

Hardmax

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - axis=1,height∗width<16384;
        - axis=3/-1,channel∗height<16384,且受限于transpose的规格限制

输出列表

- output:T1
    - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - 做hardmax的轴:1,3/-1，即channel和width方向

其他支持

- 多核联合:尚不支持

LSTM

长短期记忆网络（LSTM，Long Short-Term Memory） LSTM扩展以及变体命名为exLSTM算子，参数项中指明 （extern）的项为exLSTM独有的参数项。 计算公式如下：
Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

输入列表

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - inputsize∈(0,8192]
  - 量化支持
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]

        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- B:T2,T3
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- sequence_lens:T4,T5(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]

        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

属性列表

- direction(extern):string
    - 指定LSTM的运算方向
    - forward:指定LSTM的运算方向为前向
    - reverse:指定LSTM的运算方向为反向
    - bidirectional:指定LSTM的运算方向为双向
- sequence_size(extern):int
    - 指定LSTM输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
    - LSTM单元中的hiddensize，无限制，建议8对齐
- proj_size(extern):int
    - projection时的proj_size，projsize∈[0,hiddensize]
    - 目前限定0，即尚不支持projection功能
- input_forget(extern):int
    - cifg变种的选择:1(T)or0(F)目前限定0，即尚不支持

- has_projection(extern):int
  - projection变种:1(T)or0(F)目前限定0，即尚不支持
- input_layout(extern):int
  - 指定与对应输入shape含义一致的layout
  - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
  - snc：指定layout对应的输入shape为[seqs,batches,input_size]
  - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
  - 指定与对应输出shape含义一致的layout
  - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
  - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
  - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int32(Tensor)
- T4:float(Scalar)
- T5:int(Scalar)

其他支持

- 多核联合:暂不支持

Max

计算输入张量的元素级最大值

输入列表

- A:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Min

计算输入张量的元素级最小值

输入列表

- A:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持

        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Mul

执行元素级二进制乘法(C=A*B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

其他支持

- 多核联合:支持

MulRelu

Mul与Relu融合计算,C=Relu(A*B)

输入列表

- 同Mul

输出列表

- 同Mul

其他支持

- 多核联合:支持

Pad

给定一个包含要填充的数据的张量(data),包含轴的开始和结束填充数值的张量(pads),模式(mode),常 量数值(constant_value), 生成一个填充张量(output)

输入列表

- data:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- pads:T2
    - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end, width_end]
- constant_value(optional):T3

    - shape:[1]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

属性列表

- mode:string(默认值constant)
    - 支持模式:constant,reflect
        - constant无限制
        - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

其他支持

- 多核联合:尚不支持

MaxPool

MaxPool消耗输入张量X并根据内核大小、步幅大小和pad长度在张量上应用最大池化。最大池化包括 根据内核大小计算输入张量子集的所有值的最大值，并将数据下采样到输出张量Y中以进行进一步处理。 输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使用自 动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]
- storage_order:int64
  - 优先储存方式:0

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool使用输入张量X并对同一通道中的值应用最大池化。这相当于MaxPool的kernel_shape 大小等于input的空间维度。

输入列表

- input:T1

    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool使用输入张量X并根据内核大小、步幅大小和pad长度在张量上应用平均池化。平均池化 包括根据内核大小计算输入张量子集的所有值的平均值，并将数据下采样到输出张量Y中以进行进一步 处理。输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使 用自动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool使用输入张量X并对同一通道中的值应用平均池化。这相当于AveragePool，其kernel_shape大小等于输入张量的空间维度。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,7]
    - width∈(0,7]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z =XY 指数运算，采用输入数据（张量）和指数，并产生一个输出数据（张量）

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- Y:T2
    - shape:[1]
        - 当前仅支持0、1、2、3、0.5、-0.5

输出列表

- Z:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

其他支持

- 多核联合:尚不支持

Relu

将输入张量的负值设为零，正值保持不变

输入列表

- input:T1

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Clip/ReLU6

将输入张量的值裁剪到[0,6]范围内

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

PRelu

将输入张量的负值按可学习参数缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

属性列表

- slope:PRelu系数
    - 仅支持单个标量或C维度系数

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

LeakyRelu

将输入张量的负值按固定比例缩放，正值保持不变

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 无

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- shape:int
    - 指定输出张量的形状

其他支持

- 多核联合:尚不支持

Resize

调整输入张量的大小。一般来说，它将输出张量中的每个值计算为输入张量中邻域（即采样位置）的加 权平均值

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- roi(optional):T2
    - 目前暂不支持
- scales(optional):T2
    - 沿每个维度的缩放比例数组
- sizes(optional):T3
    - 输出张量的目标大小

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

属性列表

- antialias:int（默认值为0）

    - 目前不支持设置
- axes:int[]
    - 目前不支持设置
- coordinate_transformation_mode:strings（默认值为”half_pixel”）
    - 目前仅支持half_pixel，pytorch_half_pixel，align_corners三种
- cubic_coeff_a:float（默认为-0.75）
    - 目前不支持设置
- exclude_outside:int（默认值为0）
    - 目前不支持设置
- extrapolation_value:float（默认为0）
    - 目前不支持设置
- keep_aspect_ratio_policy:strings（默认值为”stretch”）
    - 目前不支持设置
- mode:strings（默认值为”nearest”）
    - 目前仅支持nearest和linear两种可配置
- nearest_mode:strings（默认值为”round_prefer_floor”）
    - 目前不支持设置 支持说明 目前仅支持宽高方向的整数倍缩放,且缩放倍数为2-8的整数倍,如果是(1,c,1,1)则支持放大到(1,c,h, w)
- 多核联合:不支持

Slice

获得当前向量的一个切片，具体规则和numpy切片相同

输入列表

- input_tensor:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- starts:int(Tensor)
  - 切分的起始位置:无限制
- ends:int(Tensor)
  - 切分的终止位置:无限制
- axes:int(Tensor)
  - 选取切分的轴:支持任意0～3轴，支持同时多轴选择
- steps:int(Tensor)
  - 选取切分对应轴的步长
  - 当height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]%subc==0&&steps[1]<=subc时,steps可以不为1，否则只有steps==[1,1,1,1]时，才可 通过NPU运行SliceOP

Softmax

该运算符计算给定输入的归一化指数值：

`Softmax(input, axis) = exp(input – max(input, axis)) / Σ_axis exp(input – max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - 做softmax的轴:1,3，即channel和width方向

其他支持

- 多核联合:尚不支持

---

exSoftmax13

该运算符计算给定输入的归一化指数值：

`exSoftmax13(input, axis) = exp(input - max(input, axis)) / Σ_axis exp(input - max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向

其他支持

- 多核联合:尚不支持

---

exSoftmaxMask

该运算符计算给定输入的归一化指数值：

$$\text{input}(x, mask_t, mask_value) = \begin{cases} x + (mask_t - 1) * inf & \text{if } mask_value = 0 \\ x + mask_t * (-inf) & \text{elif } mask_value = 1 \end{cases}$$

$$\text{exSoftmaxMask}(input, axis) = \frac{\exp(input - \max(input, axis))}{\sum_{axis} \exp(input - \max(input, axis))}$$

"axis"属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制
- mask(optional):T1
  - shape:[1,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,无限制; · axis=3/-1,width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向
- mask_value:int64
  - 需要mask的值：0,1

其他支持

- 多核联合:尚不支持

Split

将向量沿着axis方向分成num_outputs个向量，split用于指定切分后向量在axis方向上的大小

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
    - 一或多个输出向量，由num_outputs属性决定
    - shape:根据切分情况决定
    - 量化支持
        - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axis:int
    - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- 执行元素级二进制减法(C=A-B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - [b,c,1,1]
        - [scalar]
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - [b,c,1,1]
        - [scalar]

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

其他支持

- 多核联合:不支持

Tile

通过平铺给定的张量构造张量,这与Numpy中的函数tile相同，但没有广播

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

其他支持

- 多核联合:不支持

Transpose

对输入张量进行转置

输入列表

- X:T1
  - shape:[n1,c1,h1,w1]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

输出列表

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

属性列表

- perm转置的轴顺序: 不同perm参数限制如下：(其中p=8192,q=2048)
  - perm=[0,1,2,3]
    - 无限制
  - perm=[0,1,3,2]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q
  - perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
    - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
  - perm=[0,2,1,3]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1*w1<p
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1*w1<p
  - perm=[0,3,1,2]
    - T1,T2:h1*w1<p,w1<8192,h1*c1<p
  - perm=[0,3,2,1]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1<p*16
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1<p*8

- perm=[1,0,2,3]
  - T1:h1*w1<16*p,n1<8192,h1*w1<p
  - T2:h1*w1<8*p,n1<8192,h1*w1<p
- perm=[1,0,3,2]
  - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
  - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
- perm=[1,2,0,3]
  - T1:n1*w1<p*16,c1*h1*w1<p,w1%16=0
  - T2:n1*w1<p*8,c1*h1*w1p,w1%8=0
- perm=[1,2,3,0]
  - T1:h1*w1<p,n1<p*16,c1*h1*w1<p*16,n1*w1<p*16
  - T2:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
  - T1:c1*w1<8192,n1*h1<p,w1%16=0
  - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
  - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
  - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
  - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
  - T1:c1*h1*w1<p,n1*w1<p*16,c1<8192,n1*w1%4=0
  - T2:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
  - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
  - T1:n1*c1*h1*w1<p,h1*w1%16=0
  - T2:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
  - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%16=0
  - T2:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
  - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%16=0

        - T2:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*w1%8=0
    - perm=[3,0,1,2]
        - T1:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*h1*w1%16=0
        - T2:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*h1*w1%8=0
    - perm=[3,2,0,1]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*c1*w1<p
    - perm=[3,2,1,0]
        - T1:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*16,h1%16=0
        - T2:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*8,h1%8=0
    - perm=[3,1,0,2]
        - T1,T2:h1*w1*<p,c1*w1<8192,n1*h1<p
    - perm=[3,1,2,0]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*h1*<p,n1*c1*w1<p

数据类型约束

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Sigmoid

给定输入张量，函数y=1/(1+exp(-x))按元素应用于输入张量，得到输出张量

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

Tanh

计算给定输入张量单元的双曲正切

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

Softplus

按元素应用Softplus函数,y=ln(exp(x)+1)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

HardSigmoid

给定输入张量，函数y=max(0,min(1,alpha*x+beta))按元素应用于输入张量，得到输出张量，这里alpha=1/6，beta=0.5

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

HardSwish

按元素应用 HardSwish 函数, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x),这里alpha=1/6,beta=0.5

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

Elu

给定输入张量，函数f(x)=alpha*(exp(x)-1.)forx<0,f(x)=xforx>=0按元素应用于输入张量，得到 输出张量，这里alpha=1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exSwish

按元素应用Sigmoid线性单元函数,Swish函数也称为SiLU函数,y=x*sigmoid(x)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exMish

给定输入张量，函数mish(x)=x*tanh(softplus(x))*=x*tanh(ln(1+e^{x}))按元素应用于输入张量，得 到输出张量

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exGelu

给定输入张量，函数y =qx∗Φ(x)按元素应用于输入张量，得到输出张量，当Φ(x)函数设置成tanh时， y =0.5∗x∗(1+Tanh(2 ∗(x+0.044715∗x3))) π

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exSwooshR

给定输入张量，按论文https://arxiv.org/abs/2310.11230给定的下列函数，按元素应用于输入张量，得 到输出张量 SwooshR(x)=log(1+exp(x−1))−0.08x−0.313261687

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exSwooshL

给定输入张量，按论文https://arxiv.org/abs/2310.11230给定的下列函数，按元素应用于输入张量，得 到输出张量 SwooshL(x)=log(1+exp(x−4))−0.08x−0.035

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

Where

根据mask_tensor获取x_tensor或y_tensor的值，当mask_tensor位置为True时，取x_tensor对 应位置的值，否则取y_tensor的值。

输入列表

- mask_tensor:T1
    - shape:[batch,channel,height,width]

    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子
- x_tensor:*T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子
- y_tensor:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子

输出列表

- output:T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exConvStreaming

为了减少语音模型的时延，把原始卷积的输入分成若干个时间帧片，将每个时间帧片作为输入，计算卷 积，最后将计算结果缓存起来供后续时间帧使用，从而达到音频流式处理的目的。

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width:目前支持width=1。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exConvStreamingPRelu/LeakyRelu

exConvStreaming 与 PRelu/LeakyRelu 融合计算, Output = PRelu(exConvStreaming(Input)) 或者 Output= LeakyRelu(exConvStreaming(Input)) 说明及规格限制同exConvStreaming

---

exGlu

门控线性单元(GatedLinearUnit)函数,其中a是输入矩阵的前一半,b是后一半 GLU(a,b)=a⊗σ(b)

输入列表

- Input:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Output:T1
  - shape:[batch,channel/2,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int(默认值=1)
  - 分割输入的维度
  - 约束:仅支持1

其他支持

- 多核联合:尚不支持

---

exMatMul

矩阵乘法操作

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

输入列表

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - 量化支持
    - per-layer
- B:T1
  - shape:[b,k,1,m]
  - 量化支持
    - per-layer
- C(optional):T1,T2
  - shape:[b,n,1,m]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- Y:T1
  - shape:[b,n,1,m]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)

属性列表

- c_type:strings(默认值“add”)
  - 支持add和mul可选,表示C是做加法还是乘法

其他支持

- 多核联合:不支持

---

exNorm

对输入数据进行各种归一化操作，目前支持LayerNormalization、RMSNormalization和MeanVarianceNormalization三种：

$$\mathrm{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot W + B$$

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{RMS}^2 + \epsilon}} \cdot W + B$$

$$\mathrm{MeanVarianceNorm}(x) = \frac{x - \mu}{\sqrt{\mathrm{Variance} + \epsilon}} \cdot W + B$$

其中： x是输入向量。 μ和σ²分别是输入向量的均值和方差：

$$\mu = \frac{1}{H} \sum_{i=1}^{H} x_i$$

$$\sigma^2 = \frac{1}{H} \sum_{i=1}^{H} (x_i - \mu)^2$$

RMS是输入向量均方根：

$$\mathrm{RMS} = \sqrt{\frac{1}{H} \sum_{i=1}^{H} x_i^2}$$

Variance是输入向量的等效方差：

$$\mathrm{Variance} = \frac{1}{H} \sum_{i=1}^{H} x_i^2 - \mu^2$$

ϵ是一个很小的数，用于防止除零操作。 W 和B是可选择的参数。

输入列表

- X:T1
  - shape:[batch,channel,height,width]
- W(optional):T1
  - shape:[batch,channel,height,width]

- B(optional):T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

属性列表

- epsilon:float(默认值1e-05)
  - 用来防止除0的epsilon数值
- mode:string
  - 支持layer、rms和mean_var三种Normalization模式可选
- axes:int64[](默认值[1])
  - 归一化轴，目前只支持[1]和[1,2,3]

其他支持

- 多核联合:尚不支持

---

exSDPAttention

计算查询(query)、键(key)和值(value)之间的缩放点积注意力(SDPA)

输入列表

- query:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
  - 量化支持
    - per-layer
- key:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- value:T1
  - shape:[batch,channel,height,width]

        - channel∈(0,8192]
    - 量化支持
        - per-layer

- mask(optional):T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
- scale(optional):T3

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

其他支持

- 多核联合:不支持

---

exWindow

exWindow来源于swin_transformer的window_partition和window_reverse操作。 原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size× window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。 pytorch实现的计算方法如下：

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

输入列表

- X:T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- mode:string
  - 支持模式:partition,reverse,partition_num_first
    - partition:参考window_partition实现;
    - reverse:参考window_reverse实现;
    - partition_num_first: 与 partition 的区别在于，partition_num_first 的 H,W 按照num_windows进行划分，而partition的H,W按照window_sizes进行划分。

- window_sizes:listofints
    - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是 基于这个大小。
- num_windows:listofints
    - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。

其他支持

- 多核联合:尚不支持

RK3562

Add

执行元素级二进制加法(C=A+B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

AddRelu

Add与Relu融合计算,C=Relu(A+B)

输入列表

- 同Add

输出列表

- 同Add

其他支持

- 多核联合:支持

BatchNormalization

按照论文https://arxiv.org/abs/1502.03167中的描述对输入张量进行批量归一化计算

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- epsilon:float（默认值=1e-5）

    - 除以标准差时加上防止除0的实数
    - epsilon∈(0,∞]
- momentum:float
    - 训练时的滑动平均参数

Concat

将一系列向量在axis指定的方向上组合成一个向量，所有向量除axis指定的方向外大小必须相同

输入列表

- input_tensor:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axisint64
    - axis∈{0,1,2,3}

Convolution

卷积

输入列表

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。

    - 量化支持
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - 量化支持
        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Depthwise Convolution

深度可分离卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

ConvolutionRelu

Convolution与Relu融合计算,Output=Relu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionClip

Convolution与Clip融合计算,Output=Clip(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution 与 PRelu/LeakyRelu 融合计算, Output = PRelu(Conv(Input)) 或者 Output = LeakyRelu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionAdd

Convolution与Add融合计算,Output=Add(Conv(Input0),Input1) 说明及规格限制同Convolution

---

ConvolutionSigmoid

Convolution与Sigmoid融合计算,Output=Sigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionTanh

Convolution与Tanh融合计算,Output=Tanh(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionSoftplus

Convolution与Softplus融合计算,Output=Softplus(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSigmoid

Convolution与HardSigmoid融合计算,Output=HardSigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSwish

Convolution与HardSwish融合计算,Output=HardSwish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionElu

Convolution与Elu融合计算,Output=Elu(Conv(Input)) 说明及规格限制同Convolution

ConvolutionSwish

Convolution与Swish融合计算,Output=Swish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionMish

Convolution与Mish融合计算,Output=Mish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionAddRelu

Convolution与Add及Relu融合计算,Output=Relu(Add(Conv(Input0),Input1)) 说明及规格限制同Convolution

ConvTranspose

转置卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
- 量化支持
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,8]
  - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) 设置pad时注意: 不支持 kernel_h*dilations_h-dilations_h-pads_top<0 不支持kernel_w*dilations_w-dilations_w-pads_left<0 不支持stride_h*(height-1)-pads_top+1<output_h 不支持stride_w*(width-1)-pads_left+1<output_w
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
  - group:仅支持1,当且仅当num_input=num_output时支持num_output
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]
  - 当dilation不等于(1,1)时，只支持kernel=(1,1)

数据类型约束

- T1:int8(Tensor),float16(Tensor)

ConvTranposeRelu

ConvTranpose与Relu融合计算,Output=Relu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeClip

ConvTranpose与Clip融合计算,Output=Clip(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranpose与PRelu/LeakyRelu融合计算,Output=PRelu(ConvTranpose(Input))或者Output=LeakyRelu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeAdd

ConvTranpose与Add融合计算,Output=Add(ConvTranpose(Input0),Input1) 说明及规格限制同ConvTranspose

---

ConvTranposeSigmoid

ConvTranpose与Sigmoid融合计算,Output=Sigmoid(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeTanh

ConvTranpose与Tanh融合计算,Output=Tanh(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSoftplus

ConvTranpose与Softplus融合计算,Output=Softplus(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranpose与HardSigmoid融合计算,Output=HardSigmoid(ConvTranpose(Input))

ConvTranposeHardSwish

ConvTranpose与HardSwish融合计算,Output=HardSwish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeElu

ConvTranpose与Elu融合计算,Output=Elu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSwish

ConvTranpose与Swish融合计算,Output=Swish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeMish

ConvTranpose与Mish融合计算,Output=Mish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

Div

执行元素级二进制除法(C=A/B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

输入列表

- input1:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

广播约束

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

GRU

门控循环单元（GRU，GatedRecurrentUnit） GRU扩展以及变体命名为exGRU算子，参数项中指明（extern）的项为exGRU独有的参数项。 计算公式如下： Equations(Default:f=Sigmoid,g=Tanh): r_t = f(x_t · W_r + h_{t−1} · R_r + Wb_r + Rb_r) z_t = f(x_t · W_z + h_{t−1} · R_z + Wb_z + Rb_z) h_t = g(x_t · W_h + (r_t ⊙ h_{t−1}) · R_h + Rb_h + Wb_h), (WRB=0) h_t = g(x_t · W_h + r_t ⊙ (h_{t−1} · R_h + Rb_h)) + Wb_h, (WRB=1) H_t = (1 − z_t) ⊙ h_t + z_t ⊙ h_{t−1}

输入列表

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - inputsize∈(0,8192]
  - 量化支持
    - per-layer
- W:T1

    - shape:[num_directions,4*hidden_size,input_size]
        - inputsize∈(0,8192]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- R:T1
    - shape:[num_directions,4*hidden_size,hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- B:T2
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- sequence_lens:T3(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1

    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

属性列表

- linear_before_reset:int
    - LBR变种的选择:1(T)or0(F)
- direction(extern):string
    - 指定GRU的运算方向
    - forward:指定GRU的运算方向为前向
    - reverse:指定GRU的运算方向为反向
    - bidirectional:指定GRU的运算方向为双向
- sequence_size(extern):int
    - 指定GRU输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
    - GRU单元中的hiddensize，无限制，建议8对齐
- input_layout(extern):int
    - 指定与对应输入shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
    - snc：指定layout对应的输入shape为[seqs,batches,input_size]
    - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
    - 指定与对应输出shape含义一致的layout

    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
    - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
    - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

其他支持

- 多核联合:暂不支持

Gather

根据索引Indices获取输入X的指定axis维度的条目，并将它们拼接在一起。

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- Indices:T2
    - 要收集的元素的索引，秩rank =1

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int(Tensor)

属性列表

- axis:int(Tensor)
    - axis∈{0,1,2,3}

    - 指定Indices获取输入的维度

其他支持

- 多核联合:尚不支持

Hardmax

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - axis=1,height∗width<16384;
        - axis=3/-1,channel∗height<16384,且受限于transpose的规格限制

输出列表

- output:T1
    - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int
    - 做hardmax的轴:1,3/-1，即channel和width方向

其他支持

- 多核联合:尚不支持

LSTM

长短期记忆网络（LSTM，Long Short-Term Memory） LSTM扩展以及变体命名为exLSTM算子，参数项中指明 （extern）的项为exLSTM独有的参数项。 计算公式如下：
Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

输入列表

- X:T1
    - shape:[seq_length,batch_size,input_size]

        - seqlength∈(0,8192]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - inputsize∈(0,8192]
    - 量化支持
        - per-layer
- W:T2
    - shape:[num_directions,4*hidden_size,input_size]
        - inputsize∈(0,8192]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- R:T2
    - shape:[num_directions,4*hidden_size,hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- B:T2
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- sequence_lens:T2(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数

        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

属性列表

- direction(extern):string
  - 指定LSTM的运算方向
  - forward:指定LSTM的运算方向为前向
  - reverse:指定LSTM的运算方向为反向
  - bidirectional:指定LSTM的运算方向为双向
- sequence_size(extern):int
  - 指定LSTM输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
  - LSTM单元中的hiddensize，无限制，建议8对齐
- proj_size(extern):int
  - projection时的proj_size，projsize∈[0,hiddensize]
  - 目前限定0，即尚不支持projection功能
- input_forget(extern):int
  - cifg变种的选择:1(T)or0(F)目前限定0，即尚不支持
- has_projection(extern):int
  - projection变种:1(T)or0(F)目前限定0，即尚不支持
- input_layout(extern):int
  - 指定与对应输入shape含义一致的layout
  - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
  - snc：指定layout对应的输入shape为[seqs,batches,input_size]
  - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
  - 指定与对应输出shape含义一致的layout
  - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
  - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
  - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int32(Tensor)
- T4:float(Scalar)

其他支持

- 多核联合:暂不支持

Max

计算输入张量的元素级最大值

输入列表

- A:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Mul

执行元素级二进制乘法(C=A*B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

MulRelu

Mul与Relu融合计算,C=Relu(A*B)

输入列表

- 同Mul

输出列表

- 同Mul

其他支持

- 多核联合:支持

Pad

给定一个包含要填充的数据的张量(data),包含轴的开始和结束填充数值的张量(pads),模式(mode),常 量数值(constant_value),生成一个填充张量(output)

输入列表

- data:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

属性列表

- mode:string(默认值constant)
  - 支持模式:constant,reflect
    - constant无限制
    - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

其他支持

- 多核联合:尚不支持

MaxPool

MaxPool消耗输入张量X并根据内核大小、步幅大小和pad长度在张量上应用最大池化。最大池化包括 根据内核大小计算输入张量子集的所有值的最大值，并将数据下采样到输出张量Y中以进行进一步处理。 输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使用自 动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]

    - pad_bottom∈[0,7]
    - pad_right∈[0,7]
- storage_order:int64[]
    - 优先储存方式:0

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool使用输入张量X并对同一通道中的值应用最大池化。这相当于MaxPool的kernel_shape 大小等于input的空间维度。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]

- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool使用输入张量X并根据内核大小、步幅大小和pad长度在张量上应用平均池化。平均池化 包括根据内核大小计算输入张量子集的所有值的平均值，并将数据下采样到输出张量Y中以进行进一步 处理。输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使 用自动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64

    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool使用输入张量X并对同一通道中的值应用平均池化。这相当于AveragePool，其kernel_shape大小等于输入张量的空间维度。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]

- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z =XY 指数运算，采用输入数据（张量）和指数，并产生一个输出数据（张量）

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- Y:T2
  - shape:[1]
    - 当前仅支持0、1、2、3、0.5、-0.5

输出列表

- Z:T1
  - shape:[batch,channel,height,width]

    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

其他支持

- 多核联合:尚不支持

Relu

将输入张量的负值设为零，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Clip/ReLU6

将输入张量的值裁剪到[0,6]范围内

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 无

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

PRelu

将输入张量的负值按可学习参数缩放，正值保持不变

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer/per-channel
  - 广播支持
    - 无

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer/per-channel

属性列表

- slope:PRelu系数
    - 仅支持单个标量或C维度系数

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

LeakyRelu

将输入张量的负值按固定比例缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- shape:int
  - 指定输出张量的形状

其他支持

- 多核联合:尚不支持

Resize

调整输入张量的大小。一般来说，它将输出张量中的每个值计算为输入张量中邻域（即采样位置）的加 权平均值

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- roi(optional):T2
  - 目前暂不支持
- scales(optional):T2
  - 沿每个维度的缩放比例数组
- sizes(optional):T3

    - 输出张量的目标大小

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

属性列表

- antialias:int（默认值为0）
    - 目前不支持设置
- axes:int[]
    - 目前不支持设置
- coordinate_transformation_mode:strings（默认值为”half_pixel”）
    - 目前仅支持half_pixel，pytorch_half_pixel，align_corners三种
- cubic_coeff_a:float（默认为-0.75）
    - 目前不支持设置
- exclude_outside:int（默认值为0）
    - 目前不支持设置
- extrapolation_value:float（默认为0）
    - 目前不支持设置
- keep_aspect_ratio_policy:strings（默认值为”stretch”）
    - 目前不支持设置
- mode:strings（默认值为”nearest”）
    - 目前仅支持nearest和linear两种可配置
- nearest_mode:strings（默认值为”round_prefer_floor”）
    - 目前不支持设置 支持说明 目前仅支持宽高方向的整数倍缩放,且缩放倍数为2-8的整数倍,如果是(1,c,1,1)则支持放大到(1,c,h, w)

Slice

获得当前向量的一个切片，具体规则和numpy切片相同

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- starts:int(Tensor)
  - 切分的起始位置:无限制
- ends:int(Tensor)
  - 切分的终止位置:无限制
- axes:int(Tensor)
  - 选取切分的轴:支持任意0~3轴，支持同时多轴选择
- steps:int(Tensor)
  - 选取切分对应轴的步长
  - 当height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]%subc==0&&steps[1]<=subc时,steps可以不为1，否则只有steps==[1,1,1,1]时，才可 通过NPU运行SliceOP

Softmax

该运算符计算给定输入的归一化指数值：

`Softmax(input, axis) = exp(input - max(input, axis)) / Σ_axis exp(input - max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - 做softmax的轴:1,3，即channel和width方向

---

exSoftmax13

该运算符计算给定输入的归一化指数值：

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向

exSoftmaxMask

该运算符计算给定输入的归一化指数值：

$$\text{input}(x, mask_t, mask_value) = \begin{cases} x + (mask_t - 1) * inf & \text{if } mask_value = 0 \\ x + mask_t * (-inf) & \text{elif } mask_value = 1 \end{cases}$$

$$\text{exSoftmaxMask}(input, axis) = \frac{\exp(input - \max(input, axis))}{\sum_{axis} \exp(input - \max(input, axis))}$$

"axis"属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制
- mask(optional):T1
  - shape:[1,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,无限制; · axis=3/-1,width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向
- mask_value:int64
  - 需要mask的值：0,1

Split

将向量沿着axis方向分成num_outputs个向量，split用于指定切分后向量在axis方向上的大小

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - 一或多个输出向量，由num_outputs属性决定
  - shape:根据切分情况决定
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- 执行元素级二进制减法(C=A-B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer
    - 广播支持
        - [b,c,1,1]
        - [scalar]

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Tile

通过平铺给定的张量构造张量,这与Numpy中的函数tile相同，但没有广播

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

对输入张量进行转置

输入列表

- X:T1
  - shape:[n1,c1,h1,w1]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

输出列表

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

属性列表

- perm转置的轴顺序: 不同perm参数限制如下：(其中p=268435456,q=65536)
  - perm=[0,1,2,3]
    - 无限制
  - perm=[0,1,3,2]
    - T1:h1%16=0,h1*w1*<16p,h1*w1*<2q
    - T2:h1%8=0,h1*w1*<8p,h1*w1*<2q
  - perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
    - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
  - perm=[0,2,1,3]
    - T1:h1%16=0,h1*w1*<16p,h1*w1*<2q,c1*w1<p
    - T2:h1%8=0,h1*w1*<8p,h1*w1*<2q,c1*w1<p
  - perm=[0,3,1,2]
    - T1,T2:h1*w1*<p,w1<8192,h1*c1<p
  - perm=[0,3,2,1]
    - T1:h1%16=0,h1*w1*<16p,h1*w1*<2q,c1<p*16

    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q,c1<p*8
- perm=[1,0,2,3]
    - T1:h1*w1*<16*p,n1<8192,h1*w1<p
    - T2:h1*w1*<8*p,n1<8192,h1*w1<p
- perm=[1,0,3,2]
    - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
    - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
- perm=[1,2,0,3]
    - T1:n1*w1*<*p*16,c1*h1*w1<p,w1%16=0
    - T2:n1*w1*<*p*8,c1*h1*w1p,w1%8=0
- perm=[1,2,3,0]
    - T1:h1*w1*<*p,n1*<*p*16,c1*h1*w1<p*16,n1*w1<p*16
    - T2:h1*w1*<*p,n1*<*p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
    - T1:c1*w1*<*8192,n1*h1<p,w1%16=0
    - T2:c1*w1*<*8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
    - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
    - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
    - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
    - T1:c1*h1*w1<p,n1*w1*<*p*16,c1<8192,n1*w1%4=0
    - T2:c1*h1*w1<p,n1*w1*<*p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
    - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
    - T1:n1*c1*h1*w1*<*p,h1*w1%16=0
    - T2:n1*c1*h1*w1*<*p,h1*w1%8=0
- perm=[2,3,1,0]
    - T1:c1*h1*w1<p,n1*c1*<*p,h1*w1<8192,h1*w1%16=0
    - T2:c1*h1*w1<p,n1*c1*<*p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
    - T1,T2:h1*w1*<*8192,n1*c1<p
- perm=[3,0,2,1]

        - T1:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*w1%16=0
        - T2:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*w1%8=0
    - perm=[3,0,1,2]
        - T1:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*h1*w1%16=0
        - T2:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*h1*w1%8=0
    - perm=[3,2,0,1]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*c1*w1<p
    - perm=[3,2,1,0]
        - T1:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*16,h1%16=0
        - T2:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*8,h1%8=0
    - perm=[3,1,0,2]
        - T1,T2:h1*w1*<p,c1*w1<8192,n1*h1<p
    - perm=[3,1,2,0]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*h1*<p,n1*c1*w1<p

数据类型约束

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Sigmoid

给定输入张量，函数y=1/(1+exp(-x))按元素应用于输入张量，得到输出张量

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Tanh

计算给定输入张量单元的双曲正切

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Softplus

按元素应用Softplus函数,y=ln(exp(x)+1)

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

HardSigmoid

给定输入张量，函数y=max(0,min(1,alpha*x+beta))按元素应用于输入张量，得到输出张量，这里alpha=1/6，beta=0.5

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

HardSwish

按元素应用 HardSwish 函数, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x),这里alpha=1/6,beta=0.5

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Elu

给定输入张量，函数f(x)=alpha*(exp(x)-1.)forx<0,f(x)=xforx>=0按元素应用于输入张量，得到 输出张量，这里alpha=1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exSwish

按元素应用Sigmoid线性单元函数,Swish函数也称为SiLU函数,y=x*sigmoid(x)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exMish

给定输入张量，函数mish(x)=x*tanh(softplus(x))*=xtanh(ln(1+e^{x}))按元素应用于输入张量，得 到输出张量

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exGelu

给定输入张量，函数y =qx∗Φ(x)按元素应用于输入张量，得到输出张量，当Φ(x)函数设置成tanh时， y =0.5∗x∗(1+Tanh(2 ∗(x+0.044715∗x3))) π

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Where

根据mask_tensor获取x_tensor或y_tensor的值，当mask_tensor位置为True时，取x_tensor对 应位置的值，否则取y_tensor的值。

输入列表

- mask_tensor:T1
  - shape:[batch,channel,height,width]
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束
    - 广播约束同Expand算子
- x_tensor:*T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束
    - 广播约束同Expand算子
- y_tensor:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束
    - 广播约束同Expand算子

输出列表

- output:T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exGlu

门控线性单元(GatedLinearUnit)函数,其中a是输入矩阵的前一半,b是后一半 GLU(a,b)=a⊗σ(b)

输入列表

- Input:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Output:T1
  - shape:[batch,channel/2,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int(默认值=1)
  - 分割输入的维度
  - 约束:仅支持1

---

exMatMul

矩阵乘法操作

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

输入列表

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - 量化支持
    - per-layer

- B:T1
  - shape:[b,k,1,m]
  - 量化支持
    - per-layer
- C(optional):T1,T2
  - shape:[b,n,1,m]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- Y:T1
  - shape:[b,n,1,m]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)

属性列表

- c_type:strings(默认值“add”)
  - 支持add和mul可选,表示C是做加法还是乘法

---

exNorm

对输入数据进行各种归一化操作，目前支持LayerNormalization、RMSNormalization和MeanVarianceNormalization三种：

$$\mathrm{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot W + B$$

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{RMS}^2 + \epsilon}} \cdot W + B$$

$$\mathrm{MeanVarianceNorm}(x) = \frac{x - \mu}{\sqrt{\mathrm{Variance} + \epsilon}} \cdot W + B$$

其中： x是输入向量。 μ和σ²分别是输入向量的均值和方差：

$$\mu = \frac{1}{H}\sum_{i=1}^{H} x_i$$

$$\sigma^2 = \frac{1}{H}\sum_{i=1}^{H} (x_i - \mu)^2$$

RMS是输入向量均方根：

$$\mathrm{RMS} = \sqrt{\frac{1}{H}\sum_{i=1}^{H} x_i^2}$$

Variance是输入向量的等效方差：

$$\mathrm{Variance} = \frac{1}{H}\sum_{i=1}^{H} x_i^2 - \mu^2$$

ϵ是一个很小的数，用于防止除零操作。 W 和B是可选择的参数。

输入列表

- X:T1
  - shape:[batch,channel,height,width]
- W(optional):T1
  - shape:[batch,channel,height,width]
- B(optional):T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

属性列表

- epsilon:float(默认值1e-05)
  - 用来防止除0的epsilon数值
- mode:string
  - 支持layer、rms和mean_var三种Normalization模式可选
- axes:int64[](默认值[1])
  - 归一化轴，目前只支持[1]和[1,2,3]

其他支持

- 多核联合:尚不支持

---

exSDPAttention

计算查询(query)、键(key)和值(value)之间的缩放点积注意力(SDPA)

输入列表

- query:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
  - 量化支持
    - per-layer
- key:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- value:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
  - 量化支持
    - per-layer
- mask(optional):T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 默认支持任意维度广播操作

        - 开启FA支持后,只支持[b,c,1,1],[1,c,1,1]和[1,c,h,w]三种规格
- scale(optional):T3

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

其他支持

- FlashAttentionV2(FA):支持
    - 基于https://arxiv.org/abs/2307.08691实现,通过高速缓存内循环实现加速以及减少宽带使 用，但是会导致模型增大，请根据具体场景和模型选择是否开启使用
    - RKNPU驱动版本>=0.8.8
    - RKNPURuntime库(librknnrt.so)版本>=2.0.0
    - 支持数据类型:float16

---

exWindow

exWindow来源于swin_transformer的window_partition和window_reverse操作。 原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size× window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。 pytorch实现的计算方法如下：

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

输入列表

- X:T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- mode:string
  - 支持模式:partition,reverse,partition_num_first
    - partition:参考window_partition实现;
    - reverse:参考window_reverse实现;
    - partition_num_first: 与 partition 的区别在于，partition_num_first 的 H,W 按照num_windows进行划分，而partition的H,W按照window_sizes进行划分。

- window_sizes:listofints
    - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是 基于这个大小。
- num_windows:listofints
    - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。

其他支持

- 多核联合:尚不支持

RV1103/RV1106

Add

执行元素级二进制加法(C=A+B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(constTensor)

AddRelu

Add与Relu融合计算,C=Relu(A+B)

输入列表

- 同Add

输出列表

- 同Add

其他支持

- 多核联合:支持

BatchNormalization

按照论文https://arxiv.org/abs/1502.03167中的描述对输入张量进行批量归一化计算

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- epsilon:float（默认值=1e-5）

    - 除以标准差时加上防止除0的实数
    - epsilon∈(0,∞]
- momentum:float
    - 训练时的滑动平均参数

Concat

将一系列向量在axis指定的方向上组合成一个向量，所有向量除axis指定的方向外大小必须相同

输入列表

- input_tensor:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axisint64
    - axis∈{0,1,2,3}

Convolution

卷积

输入列表

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。

    - 量化支持
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - 量化支持
        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

数据类型约束

- T1:int8(Tensor)

Depthwise Convolution

深度可分离卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

数据类型约束

- T1:int8(Tensor)

ConvolutionRelu

Convolution与Relu融合计算,Output=Relu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionClip

Convolution与Clip融合计算,Output=Clip(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution 与 PRelu/LeakyRelu 融合计算, Output = PRelu(Conv(Input)) 或者 Output = LeakyRelu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionAdd

Convolution与Add融合计算,Output=Add(Conv(Input0),Input1) 说明及规格限制同Convolution

---

ConvolutionSigmoid

Convolution与Sigmoid融合计算,Output=Sigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionTanh

Convolution与Tanh融合计算,Output=Tanh(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionSoftplus

Convolution与Softplus融合计算,Output=Softplus(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSigmoid

Convolution与HardSigmoid融合计算,Output=HardSigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSwish

Convolution与HardSwish融合计算,Output=HardSwish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionElu

Convolution与Elu融合计算,Output=Elu(Conv(Input)) 说明及规格限制同Convolution

ConvolutionSwish

Convolution与Swish融合计算,Output=Swish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionMish

Convolution与Mish融合计算,Output=Mish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionAddRelu

Convolution与Add及Relu融合计算,Output=Relu(Add(Conv(Input0),Input1)) 说明及规格限制同Convolution

ConvTranspose

转置卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
- 量化支持
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,8]
  - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) 设置pad时注意: 不支持 kernel_h*dilations_h-dilations_h-pads_top<0 不支持kernel_w*dilations_w-dilations_w-pads_left<0 不支持stride_h*(height-1)-pads_top+1<output_h 不支持stride_w*(width-1)-pads_left+1<output_w
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
  - group:仅支持1,当且仅当num_input=num_output时支持num_output
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]
  - 当dilation不等于(1,1)时，只支持kernel=(1,1)

数据类型约束

- T1:int8(Tensor)

ConvTranposeRelu

ConvTranpose与Relu融合计算,Output=Relu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeClip

ConvTranpose与Clip融合计算,Output=Clip(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranpose与PRelu/LeakyRelu融合计算,Output=PRelu(ConvTranpose(Input))或者Output=LeakyRelu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeAdd

ConvTranpose与Add融合计算,Output=Add(ConvTranpose(Input0),Input1) 说明及规格限制同ConvTranspose

---

ConvTranposeSigmoid

ConvTranpose与Sigmoid融合计算,Output=Sigmoid(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeTanh

ConvTranpose与Tanh融合计算,Output=Tanh(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSoftplus

ConvTranpose与Softplus融合计算,Output=Softplus(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranpose与HardSigmoid融合计算,Output=HardSigmoid(ConvTranpose(Input))

ConvTranposeHardSwish

ConvTranpose与HardSwish融合计算,Output=HardSwish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeElu

ConvTranpose与Elu融合计算,Output=Elu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSwish

ConvTranpose与Swish融合计算,Output=Swish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeMish

ConvTranpose与Mish融合计算,Output=Mish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

Div

执行元素级二进制除法(C=A/B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

根据给定的 shape 和广播规则广播输入张量。广播规则类似于 numpy.array(input) * numpy.ones(shape)：维度右对齐；两个对应维度必须具有相同的值，或者其中一个等于1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- shape:T2
  - shape:[batch,channel,height,width]

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:int64(Tensor)

广播约束

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

Hardmax

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∗height∗width<=8192
    - axis=1,height∗width<16384且height∗width等于1或大于8;
    - axis=3/-1,channel ∗ height < 16384, 且 channel ∗ height 等于 1 或大于 8，且受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - 做hardmax的轴:1,3/-1，即channel和width方向

其他支持

- 多核联合:尚不支持

LSTM

长短期记忆网络（LSTM，Long Short-Term Memory） LSTM扩展以及变体命名为exLSTM算子，参数项中指明 （extern）的项为exLSTM独有的参数项。 计算公式如下：
Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

输入列表

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - inputsize∈(0,8192]
  - 量化支持
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer

        - per-channel
- R:T1
    - shape:[num_directions,4*hidden_size,hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- B:T2
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- sequence_lens:T3(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- initial_c:T3(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1
  - shape:[seq_length,num_directions,batch_size,hidden_size]
    - seqlength∈(0,8192]
    - numdirections:1or2
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - hiddensize∈(0,8192]
  - 量化支持
    - per-layer
- Y_h:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]
    - numdirections:1or2
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - hiddensize∈(0,8192]
  - 量化支持
    - per-layer
- Y_c:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]
    - numdirections:1or2
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - hiddensize∈(0,8192]
  - 量化支持
    - per-layer

属性列表

- direction(extern):string
  - 指定LSTM的运算方向
  - forward:指定LSTM的运算方向为前向
  - reverse:指定LSTM的运算方向为反向
  - bidirectional:指定LSTM的运算方向为双向
- sequence_size(extern):int
  - 指定LSTM输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
  - LSTM单元中的hiddensize，无限制，建议8对齐
- proj_size(extern):int
  - projection时的proj_size，projsize∈[0,hiddensize]

    - 目前限定0，即尚不支持projection功能
- input_forget(extern):int
    - cifg变种的选择:1(T)or0(F)目前限定0，即尚不支持
- has_projection(extern):int
    - projection变种:1(T)or0(F)目前限定0，即尚不支持
- input_layout(extern):int
    - 指定与对应输入shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
    - snc：指定layout对应的输入shape为[seqs,batches,input_size]
    - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
    - 指定与对应输出shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
    - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
    - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:int8(Tensor)
- T2:int32(Tensor)
- T3:int(Scalar)

其他支持

- 多核联合:暂不支持

Max

计算输入张量的元素级最大值

输入列表

- A:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

- B:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Min

计算输入张量的元素级最小值

输入列表

- A:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Mul

执行元素级二进制乘法(C=A*B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(constTensor)

MulRelu

Mul与Relu融合计算,C=Relu(A*B)

输入列表

- 同Mul

输出列表

- 同Mul

其他支持

- 多核联合:支持

Pad

给定一个包含要填充的数据的张量(data),包含轴的开始和结束填充数值的张量(pads),模式(mode),常 量数值(constant_value),生成一个填充张量(output)

输入列表

- data:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- pads:T2
    - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
    - shape:[1]
    - 量化支持

        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

属性列表

- mode:string(默认值constant)
    - 支持模式:constant,reflect
        - constant无限制
        - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

其他支持

- 多核联合:尚不支持

MaxPool

MaxPool消耗输入张量X并根据内核大小、步幅大小和pad长度在张量上应用最大池化。最大池化包括 根据内核大小计算输入张量子集的所有值的最大值，并将数据下采样到输出张量Y中以进行进一步处理。 输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使用自 动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

输出列表

- output:T1

    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]
- storage_order:int64[]
    - 优先储存方式:0

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool使用输入张量X并对同一通道中的值应用最大池化。这相当于MaxPool的kernel_shape 大小等于input的空间维度。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]

    - height∈(0,7]
    - width∈(0,7]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool使用输入张量X并根据内核大小、步幅大小和pad长度在张量上应用平均池化。平均池化 包括根据内核大小计算输入张量子集的所有值的平均值，并将数据下采样到输出张量Y中以进行进一步 处理。输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使 用自动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool使用输入张量X并对同一通道中的值应用平均池化。这相当于AveragePool，其kernel_shape大小等于输入张量的空间维度。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,7]
    - width∈(0,7]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z =XY 指数运算，采用输入数据（张量）和指数，并产生一个输出数据（张量）

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- Y:T2
    - shape:[1]
        - 当前仅支持0、1、2、3、0.5、-0.5

输出列表

- Z:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

其他支持

- 多核联合:尚不支持

Relu

将输入张量的负值设为零，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持

        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Clip/ReLU6

将输入张量的值裁剪到[0,6]范围内

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

PRelu

将输入张量的负值按可学习参数缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

属性列表

- slope:PRelu系数
    - 仅支持单个标量或C维度系数

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

LeakyRelu

将输入张量的负值按固定比例缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]

    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- shape:int
    - 指定输出张量的形状

其他支持

- 多核联合:尚不支持

Resize

调整输入张量的大小。一般来说，它将输出张量中的每个值计算为输入张量中邻域（即采样位置）的加 权平均值

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- roi(optional):T2
    - 目前暂不支持
- scales(optional):T2
    - 沿每个维度的缩放比例数组
- sizes(optional):T3
    - 输出张量的目标大小

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

属性列表

- antialias:int（默认值为0）
    - 目前不支持设置

- axes:int[]
  - 目前不支持设置
- coordinate_transformation_mode:strings（默认值为”half_pixel”）
  - 目前仅支持half_pixel，pytorch_half_pixel，align_corners三种
- cubic_coeff_a:float（默认为-0.75）
  - 目前不支持设置
- exclude_outside:int（默认值为0）
  - 目前不支持设置
- extrapolation_value:float（默认为0）
  - 目前不支持设置
- keep_aspect_ratio_policy:strings（默认值为”stretch”）
  - 目前不支持设置
- mode:strings（默认值为”nearest”）
  - 目前仅支持nearest和linear两种可配置
- nearest_mode:strings（默认值为”round_prefer_floor”）
  - 目前不支持设置 支持说明 目前仅支持宽高方向的整数倍缩放,且缩放倍数为2-8的整数倍,如果是(1,c,1,1)则支持放大到(1,c,h, w)

Slice

获得当前向量的一个切片，具体规则和numpy切片相同

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- starts:int(Tensor)
  - 切分的起始位置:无限制
- ends:int(Tensor)
  - 切分的终止位置:无限制
- axes:int(Tensor)
  - 选取切分的轴:支持任意0~3轴，支持同时多轴选择
- steps:int(Tensor)
  - 选取切分对应轴的步长
  - 当height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]%subc==0&&steps[1]<=subc时,steps可以不为1，否则只有steps==[1,1,1,1]时，才可 通过NPU运行SliceOP

Softmax

该运算符计算给定输入的归一化指数值：

`Softmax(input, axis) = exp(input – max(input, axis)) / Σ_axis exp(input – max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - 做softmax的轴:1,3，即channel和width方向

---

exSoftmax13

该运算符计算给定输入的归一化指数值：

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向

---

exSoftmaxMask

该运算符计算给定输入的归一化指数值：

$$\text{input}(x, mask_t, mask_value) = \begin{cases} x + (mask_t - 1) * inf & \text{if } mask_value = 0 \\ x + mask_t * (-inf) & \text{elif } mask_value = 1 \end{cases}$$

$$\text{exSoftmaxMask}(input, axis) = \frac{\exp(input - \max(input, axis))}{\sum_{axis} \exp(input - \max(input, axis))}$$

"axis"属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

- mask(optional):T1
  - shape:[1,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,无限制; · axis=3/-1,width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向
- mask_value:int64
  - 需要mask的值：0,1

Split

将向量沿着axis方向分成num_outputs个向量，split用于指定切分后向量在axis方向上的大小

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - 一或多个输出向量，由num_outputs属性决定
  - shape:根据切分情况决定
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- 执行元素级二进制减法(C=A-B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - [b,c,1,1]
    - [scalar]

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(constTensor)

Tile

通过平铺给定的张量构造张量,这与Numpy中的函数tile相同，但没有广播

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- repeats:T2
  - shape:[batch,channel,height,width]

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

对输入张量进行转置

输入列表

- X:T1
  - shape:[n1,c1,h1,w1]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

输出列表

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - 详细约束规则见属性列表

    - 量化支持
        - per-layer

属性列表

- perm转置的轴顺序: 不同perm参数限制如下：(其中p=8192,q=2048)
    - perm=[0,1,2,3]
        - 无限制
    - perm=[0,1,3,2]
        - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q
        - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q
    - perm=[0,2,3,1]
        - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
        - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
    - perm=[0,2,1,3]
        - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1*w1<p
        - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1*w1<p
    - perm=[0,3,1,2]
        - T1,T2:h1*w1<p,w1<8192,h1*c1<p
    - perm=[0,3,2,1]
        - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1<p*16
        - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1<p*8
    - perm=[1,0,2,3]
        - T1:h1*w1<16*p,n1<8192,h1*w1<p
        - T2:h1*w1<8*p,n1<8192,h1*w1<p
    - perm=[1,0,3,2]
        - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
        - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
    - perm=[1,2,0,3]
        - T1:n1*w1<p*16,c1*h1*w1<p,w1%16=0
        - T2:n1*w1<p*8,c1*h1*w1p,w1%8=0
    - perm=[1,2,3,0]
        - T1:h1*w1<p,n1<p*16,c1*h1*w1<p*16,n1*w1<p*16
        - T2:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
    - perm=[1,3,0,2]
        - T1:c1*w1<8192,n1*h1<p,w1%16=0

    - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
    - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
    - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
    - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
    - T1:c1*h1*w1<p,n1*w1<p*16,c1<8192,n1*w1%4=0
    - T2:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
    - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
    - T1:n1*c1*h1*w1<p,h1*w1%16=0
    - T2:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
    - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%16=0
    - T2:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
    - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
    - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%16=0
    - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%8=0
- perm=[3,0,1,2]
    - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%16=0
    - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%8=0
- perm=[3,2,0,1]
    - T1,T2:h1*w1<p,n1*c1<8192,n1*c1*w1<p
- perm=[3,2,1,0]
    - T1:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*16,h1%16=0
    - T2:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*8,h1%8=0
- perm=[3,1,0,2]
    - T1,T2:h1*w1<p,c1*w1<8192,n1*h1<p
- perm=[3,1,2,0]
    - T1,T2:h1*w1<p,n1*c1<8192,n1*h1<p,n1*c1*w1<p

数据类型约束

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Sigmoid

给定输入张量，函数y=1/(1+exp(-x))按元素应用于输入张量，得到输出张量

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Tanh

计算给定输入张量单元的双曲正切

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Softplus

按元素应用Softplus函数,y=ln(exp(x)+1)

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

HardSigmoid

给定输入张量，函数y=max(0,min(1,alpha*x+beta))按元素应用于输入张量，得到输出张量，这里alpha=1/6，beta=0.5

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

HardSwish

按元素应用 HardSwish 函数, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x),这里alpha=1/6,beta=0.5

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Elu

给定输入张量，函数f(x)=alpha*(exp(x)-1.)forx<0,f(x)=xforx>=0按元素应用于输入张量，得到 输出张量，这里alpha=1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exSwish

按元素应用Sigmoid线性单元函数,Swish函数也称为SiLU函数,y=x*sigmoid(x)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

---

exMish

给定输入张量，函数mish(x)=x*tanh(softplus(x))*=xtanh(ln(1+e^{x}))按元素应用于输入张量，得 到输出张量

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

exGelu

给定输入张量，函数y =qx∗Φ(x)按元素应用于输入张量，得到输出张量，当Φ(x)函数设置成tanh时， y =0.5∗x∗(1+Tanh(2 ∗(x+0.044715∗x3))) π

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

Where

根据mask_tensor获取x_tensor或y_tensor的值，当mask_tensor位置为True时，取x_tensor对 应位置的值，否则取y_tensor的值。

输入列表

- mask_tensor:T1
  - shape:[batch,channel,height,width]
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束
    - 广播约束同Expand算子
- x_tensor:*T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持

        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子
- y_tensor:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子

输出列表

- output:T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exGlu

门控线性单元(GatedLinearUnit)函数,其中a是输入矩阵的前一半,b是后一半 GLU(a,b)=a⊗σ(b)

输入列表

- Input:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Output:T1
    - shape:[batch,channel/2,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int(默认值=1)
  - 分割输入的维度
  - 约束:仅支持1

---

exMatMul

矩阵乘法操作

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

输入列表

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - 量化支持
    - per-layer
- B:T1
  - shape:[b,k,1,m]
  - 量化支持
    - per-layer
- C(optional):T1,T2
  - shape:[b,n,1,m]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- Y:T1
  - shape:[b,n,1,m]
  - 量化支持

        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(Tensor)

属性列表

- c_type:strings(默认值“add”)
    - 支持add和mul可选,表示C是做加法还是乘法

---

exSDPAttention

计算查询(query)、键(key)和值(value)之间的缩放点积注意力(SDPA)

输入列表

- query:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
    - 量化支持
        - per-layer
- key:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- value:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
    - 量化支持
        - per-layer
- mask(optional):T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 默认支持任意维度广播操作

- scale(optional):T3

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

---

exWindow

exWindow来源于[swin_transformer]的window_partition和window_reverse操作。 原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size× window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。 pytorch实现的计算方法如下：

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

输入列表

- X:T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- mode:string
  - 支持模式:partition,reverse,partition_num_first
    - partition:参考window_partition实现;
    - reverse:参考window_reverse实现;
    - partition_num_first: 与 partition 的区别在于，partition_num_first 的 H,W 按照num_windows进行划分，而partition的H,W按照window_sizes进行划分。
- window_sizes:listofints
  - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是 基于这个大小。
- num_windows:listofints
  - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。

其他支持

- 多核联合:尚不支持

RK3576/RV1126B

Add

执行元素级二进制加法(C=A+B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

其他支持

- 多核联合:RK3576支持/RV1126B不支持

AddRelu

Add与Relu融合计算,C=Relu(A+B)

输入列表

- 同Add

输出列表

- 同Add

其他支持

- 多核联合:RK3576支持/RV1126B不支持

BatchNormalization

按照论文https://arxiv.org/abs/1502.03167中的描述对输入张量进行批量归一化计算

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- epsilon:float（默认值=1e-5）
    - 除以标准差时加上防止除0的实数
    - epsilon∈(0,∞]
- momentum:float
    - 训练时的滑动平均参数

其他支持

- 多核联合:尚不支持

Concat

将一系列向量在axis指定的方向上组合成一个向量，所有向量除axis指定的方向外大小必须相同

输入列表

- input_tensor:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axisint64
  - axis∈{0,1,2,3}

Convolution

卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:RK3576支持/RV1126B不支持

Depthwise Convolution

深度可分离卷积

输入列表

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
    - 量化支持
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,8]
        - kernel_w∈[1,8]

    - 量化支持
        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionRelu

Convolution与Relu融合计算,Output=Relu(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionClip

Convolution与Clip融合计算,Output=Clip(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionPRelu/LeakyRelu

Convolution 与 PRelu/LeakyRelu 融合计算, Output = PRelu(Conv(Input)) 或者 Output = LeakyRelu(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionAdd

Convolution与Add融合计算,Output=Add(Conv(Input0),Input1) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionSigmoid

Convolution与Sigmoid融合计算,Output=Sigmoid(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionTanh

Convolution与Tanh融合计算,Output=Tanh(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionSoftplus

Convolution与Softplus融合计算,Output=Softplus(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionHardSigmoid

Convolution与HardSigmoid融合计算,Output=HardSigmoid(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionHardSwish

Convolution与HardSwish融合计算,Output=HardSwish(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionElu

Convolution与Elu融合计算,Output=Elu(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionSwish

Convolution与Swish融合计算,Output=Swish(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionMish

Convolution与Mish融合计算,Output=Mish(Conv(Input)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvolutionAddRelu

Convolution与Add及Relu融合计算,Output=Relu(Add(Conv(Input0),Input1)) 说明及规格限制同Convolution

其他支持

- 多核联合:RK3576支持/RV1126B不支持

ConvTranspose

转置卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,8]
  - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) 设置pad时注意: 不支持 kernel_h*dilations_h-dilations_h-pads_top<0 不支持kernel_w*dilations_w-dilations_w-pads_left<0 不支持stride_h*(height-1)-pads_top+1<output_h 不支持stride_w*(width-1)-pads_left+1<output_w
  - pads_top∈[0,15]
  - pads_left∈[0,15]

    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group:仅支持1,当且仅当num_input=num_output时支持num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - 当dilation不等于(1,1)时，只支持kernel=(1,1)

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

ConvTranposeRelu

ConvTranpose与Relu融合计算,Output=Relu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeClip

ConvTranpose与Clip融合计算,Output=Clip(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranpose与PRelu/LeakyRelu融合计算,Output=PRelu(ConvTranpose(Input))或者Output=LeakyRelu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeAdd

ConvTranpose与Add融合计算,Output=Add(ConvTranpose(Input0),Input1) 说明及规格限制同ConvTranspose

---

ConvTranposeSigmoid

ConvTranpose与Sigmoid融合计算,Output=Sigmoid(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeTanh

ConvTranpose与Tanh融合计算,Output=Tanh(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSoftplus

ConvTranpose与Softplus融合计算,Output=Softplus(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranpose与HardSigmoid融合计算,Output=HardSigmoid(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeHardSwish

ConvTranpose与HardSwish融合计算,Output=HardSwish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeElu

ConvTranpose与Elu融合计算,Output=Elu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeSwish

ConvTranpose与Swish融合计算,Output=Swish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeMish

ConvTranpose与Mish融合计算,Output=Mish(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

Div

执行元素级二进制除法(C=A/B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

其他支持

- 多核联合:不支持

Expand

根据给定的 shape 和广播规则广播输入张量。广播规则类似于 numpy.array(input) * numpy.ones(shape)：维度右对齐；两个对应维度必须具有相同的值，或者其中一个等于1

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- shape:T2

    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

广播约束

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

其他支持

- 多核联合:不支持

GRU

门控循环单元（GRU，GatedRecurrentUnit） GRU扩展以及变体命名为exGRU算子，参数项中指明（extern）的项为exGRU独有的参数项。 计算公式如下： Equations(Default:f=Sigmoid,g=Tanh): r_t = f(x_t · W_r + h_{t−1} · R_r + Wb_r + Rb_r) z_t = f(x_t · W_z + h_{t−1} · R_z + Wb_z + Rb_z) h_t = g(x_t · W_h + (r_t ⊙ h_{t−1}) · R_h + Rb_h + Wb_h), (WRB=0) h_t = g(x_t · W_h + r_t ⊙ (h_{t−1} · R_h + Rb_h)) + Wb_h, (WRB=1) H_t = (1 − z_t) ⊙ h_t + z_t ⊙ h_{t−1}

输入列表

- X:T1
    - shape:[seq_length,batch_size,input_size]
        - seqlength∈(0,8192]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - inputsize∈(0,8192]
    - 量化支持
        - per-layer

- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- B:T2
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- sequence_lens:T3(optional)
  - shape:[batch_size]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]
    - numdirections:1or2
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - hiddensize∈(0,8192]
  - 量化支持
    - per-layer

输出列表

- Y:T1
  - shape:[seq_length,num_directions,batch_size,hidden_size]
    - seqlength∈(0,8192]
    - numdirections:1or2
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - hiddensize∈(0,8192]
  - 量化支持
    - per-layer
- Y_h:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]
    - numdirections:1or2
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - hiddensize∈(0,8192]
  - 量化支持
    - per-layer

属性列表

- linear_before_reset:int
  - LBR变种的选择:1(T)or0(F)
- direction(extern):string
  - 指定GRU的运算方向
  - forward:指定GRU的运算方向为前向
  - reverse:指定GRU的运算方向为反向
  - bidirectional:指定GRU的运算方向为双向
- sequence_size(extern):int
  - 指定GRU输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
  - GRU单元中的hiddensize，无限制，建议8对齐
- input_layout(extern):int
  - 指定与对应输入shape含义一致的layout
  - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
  - snc：指定layout对应的输入shape为[seqs,batches,input_size]
  - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int

    - 指定与对应输出shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
    - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
    - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

其他支持

- 多核联合:暂不支持

Gather

根据索引Indices获取输入X的指定axis维度的条目，并将它们拼接在一起。

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- Indices:T2
    - 要收集的元素的索引，秩rank =1

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int(Tensor)

属性列表

- axis:int(Tensor)

- axis∈{0,1,2,3}
- 指定Indices获取输入的维度

其他支持

- 多核联合:尚不支持

Hardmax

该OP计算给定输入的hardmax激活函数，实现参照onnx中的Hardmax-13

$$\text{Hardmax}(input, axis) = \begin{cases} 1 & \text{if } i = \arg\max(input, axis) \\ 0 & \text{otherwise} \end{cases}$$

输入列表

- input:T
  - shape:[batch,channel,height,width]
    - axis=1时,height∗width∈[1,16384);
    - axis=3/-1时,channel∗height∈[1,16384),且受限于transpose的规格限制

输出列表

- output:T
  - shape:[batch,channel,height,width]

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - 做hardmax的轴:axis∈{−1,1,3}即channel和width方向

LSTM

长短期记忆网络（LSTM，Long Short-Term Memory） LSTM扩展以及变体命名为exLSTM算子，参数项中指明 （extern）的项为exLSTM独有的参数项。 计算公式如下：
Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

输入列表

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - inputsize∈(0,8192]
  - 量化支持
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- B:T2,T3
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- sequence_lens:T4,T5(optional)
  - shape:[batch_size]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]

        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持

    - per-layer

属性列表

- direction(extern):string
    - 指定LSTM的运算方向
    - forward:指定LSTM的运算方向为前向
    - reverse:指定LSTM的运算方向为反向
    - bidirectional:指定LSTM的运算方向为双向
- sequence_size(extern):int
    - 指定LSTM输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
    - LSTM单元中的hiddensize，无限制，建议8对齐
- proj_size(extern):int
    - projection时的proj_size，projsize∈[0,hiddensize]
    - 目前限定0，即尚不支持projection功能
- input_forget(extern):int
    - cifg变种的选择:1(T)or0(F)目前限定0，即尚不支持
- has_projection(extern):int
    - projection变种:1(T)or0(F)目前限定0，即尚不支持
- input_layout(extern):int
    - 指定与对应输入shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
    - snc：指定layout对应的输入shape为[seqs,batches,input_size]
    - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
    - 指定与对应输出shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
    - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
    - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int32(Tensor)

- T4:float(Scalar)
- T5:int(Scalar)

其他支持

- 多核联合:暂不支持

Max

计算输入张量的元素级最大值

输入列表

- A:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Greater

计算输入张量A和B的Greater结果

输入列表

- A:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Min

计算输入张量的元素级最小值

输入列表

- A:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer/per-channel

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Mul

执行元素级二进制乘法(C=A*B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

        - 广播支持
            - 支持任意维度广播操作
            - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

其他支持

- 多核联合:RK3576支持/RV1126B不支持

MulRelu

Mul与Relu融合计算,C=Relu(A*B)

输入列表

- 同Mul

输出列表

- 同Mul

其他支持

- 多核联合:RK3576支持/RV1126B不支持

Pad

给定一个包含要填充的数据的张量(data),包含轴的开始和结束填充数值的张量(pads),模式(mode),常 量数值(constant_value),生成一个填充张量(output)

输入列表

- data:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

属性列表

- mode:string(默认值constant)
  - 支持模式:constant,reflect
    - constant无限制
    - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

其他支持

- 多核联合:尚不支持

MaxPool

MaxPool消耗输入张量X并根据内核大小、步幅大小和pad长度在张量上应用最大池化。最大池化包括 根据内核大小计算输入张量子集的所有值的最大值，并将数据下采样到输出张量Y中以进行进一步处理。 输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使用自 动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]
- storage_order:int64[]

    - 优先储存方式:0

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool使用输入张量X并对同一通道中的值应用最大池化。这相当于MaxPool的kernel_shape 大小等于input的空间维度。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)

    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool使用输入张量X并根据内核大小、步幅大小和pad长度在张量上应用平均池化。平均池化 包括根据内核大小计算输入张量子集的所有值的平均值，并将数据下采样到输出张量Y中以进行进一步 处理。输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使 用自动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]

    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool使用输入张量X并对同一通道中的值应用平均池化。这相当于AveragePool，其kernel_shape大小等于输入张量的空间维度。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(1,7]
        - width∈(1,7]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64

    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z =XY 指数运算，采用输入数据（张量）和指数，并产生一个输出数据（张量）

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- Y:T2
    - shape:[1]
        - 当前仅支持0、1、2、3、0.5、-0.5

输出列表

- Z:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

其他支持

- 多核联合:尚不支持

ReduceMax

获取输入张量channel方向上的最大值

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,1,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:暂不支持

Relu

将输入张量的负值设为零，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持

        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Clip/ReLU6

将输入张量的值裁剪到[0,6]范围内

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

PRelu

将输入张量的负值按可学习参数缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

属性列表

- slope:PRelu系数
    - 仅支持单个标量或C维度系数

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

LeakyRelu

将输入张量的负值按固定比例缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]

    - 量化支持
        - per-layer
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- shape:int
    - 指定输出张量的形状

其他支持

- 多核联合:尚不支持

Resize

调整输入张量的大小。一般来说，它将输出张量中的每个值计算为输入张量中邻域（即采样位置）的加 权平均值

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- roi(optional):T2
    - 目前暂不支持
- scales(optional):T2
    - 沿每个维度的缩放比例数组
- sizes(optional):T3
    - 输出张量的目标大小

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

属性列表

- antialias:int（默认值为0）
    - 目前不支持设置

- axes:int[]
  - 目前不支持设置
- coordinate_transformation_mode:strings（默认值为”half_pixel”）
  - 目前仅支持half_pixel，pytorch_half_pixel，align_corners三种
- cubic_coeff_a:float（默认为-0.75）
  - 目前不支持设置
- exclude_outside:int（默认值为0）
  - 目前不支持设置
- extrapolation_value:float（默认为0）
  - 目前不支持设置
- keep_aspect_ratio_policy:strings（默认值为”stretch”）
  - 目前不支持设置
- mode:strings（默认值为”nearest”）
  - 目前仅支持nearest和linear两种可配置
- nearest_mode:strings（默认值为”round_prefer_floor”）
  - 目前不支持设置 支持说明 目前仅支持宽高方向的整数倍缩放,且缩放倍数为2-8的整数倍,如果是(1,c,1,1)则支持放大到(1,c,h, w)
- 多核联合:不支持

Slice

获得当前向量的一个切片，具体规则和numpy切片相同

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- starts:int(Tensor)
  - 切分的起始位置:无限制
- ends:int(Tensor)
  - 切分的终止位置:无限制
- axes:int(Tensor)
  - 选取切分的轴:支持任意0~3轴，支持同时多轴选择
- steps:int(Tensor)
  - 选取切分对应轴的步长
  - 当height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]%subc==0&&steps[1]<=subc时,steps可以不为1，否则只有steps==[1,1,1,1]时，才可 通过NPU运行SliceOP

Softmax

该运算符计算给定输入的归一化指数值：

`Softmax(input, axis) = exp(input – max(input, axis)) / Σ_axis exp(input – max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向

其他支持

- 多核联合:尚不支持

---

exSoftmax13

该运算符计算给定输入的归一化指数值：

`exSoftmax13(input, axis) = exp(input - max(input, axis)) / Σ_axis exp(input - max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向

其他支持

- 多核联合:尚不支持

---

exSoftmaxMask

该运算符计算给定输入的归一化指数值：

$$\text{input}(x, mask_t, mask_value) = \begin{cases} x + (mask_t - 1) * inf & \text{if } mask_value = 0 \\ x + mask_t * (-inf) & \text{elif } mask_value = 1 \end{cases}$$

$$\text{exSoftmaxMask}(input, axis) = \frac{\exp(input - \max(input, axis))}{\sum_{axis} \exp(input - \max(input, axis))}$$

"axis"属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
    - 受限于transpose的规格限制
- mask(optional):T1
  - shape:[1,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,无限制; · axis=3/-1,width∈(0,8192]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int64
  - 做softmax的轴:1,3，即channel和width方向
- mask_value:int64
  - 需要mask的值：0,1

其他支持

- 多核联合:尚不支持

Split

将向量沿着axis方向分成num_outputs个向量，split用于指定切分后向量在axis方向上的大小

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - 一或多个输出向量，由num_outputs属性决定
  - shape:根据切分情况决定
  - 量化支持
    - per-layer

数据类型约束

- T:int8(Tensor),float16(Tensor)

属性列表

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- 执行元素级二进制减法(C=A-B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - [b,c,1,1]
    - [scalar]

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

其他支持

- 多核联合:不支持

Tile

通过平铺给定的张量构造张量,这与Numpy中的函数tile相同，但没有广播

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

其他支持

- 多核联合:不支持

Transpose

对输入张量进行转置

输入列表

- X:T1
  - shape:[n1,c1,h1,w1]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

输出列表

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

属性列表

- perm转置的轴顺序: 不同perm参数限制如下：(其中p=268435456,q=65536)
  - perm=[0,1,2,3]
    - 无限制
  - perm=[0,1,3,2]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q
  - perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
    - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
  - perm=[0,2,1,3]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1*w1<p
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1*w1<p
  - perm=[0,3,1,2]
    - T1,T2:h1*w1<p,w1<8192,h1*c1<p
  - perm=[0,3,2,1]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1<p*16
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1<p*8

- perm=[1,0,2,3]
  - T1:h1*w1<16*p,n1<8192,h1*w1<p
  - T2:h1*w1<8*p,n1<8192,h1*w1<p
- perm=[1,0,3,2]
  - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
  - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
- perm=[1,2,0,3]
  - T1:n1*w1<p*16,c1*h1*w1<p,w1%16=0
  - T2:n1*w1<p*8,c1*h1*w1p,w1%8=0
- perm=[1,2,3,0]
  - T1:h1*w1<p,n1<p*16,c1*h1*w1<p*16,n1*w1<p*16
  - T2:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
  - T1:c1*w1<8192,n1*h1<p,w1%16=0
  - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
  - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
  - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
  - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
  - T1:c1*h1*w1<p,n1*w1<p*16,c1<8192,n1*w1%4=0
  - T2:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
  - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
  - T1:n1*c1*h1*w1<p,h1*w1%16=0
  - T2:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
  - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%16=0
  - T2:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
  - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%16=0

        - T2:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*w1%8=0
    - perm=[3,0,1,2]
        - T1:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*h1*w1%16=0
        - T2:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*h1*w1%8=0
    - perm=[3,2,0,1]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*c1*w1<p
    - perm=[3,2,1,0]
        - T1:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*16,h1%16=0
        - T2:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*8,h1%8=0
    - perm=[3,1,0,2]
        - T1,T2:h1*w1*<p,c1*w1<8192,n1*h1<p
    - perm=[3,1,2,0]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*h1*<p,n1*c1*w1<p

数据类型约束

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Sigmoid

给定输入张量，函数y=1/(1+exp(-x))按元素应用于输入张量，得到输出张量

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

Tanh

计算给定输入张量单元的双曲正切

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

Softplus

按元素应用Softplus函数,y=ln(exp(x)+1)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

HardSigmoid

给定输入张量，函数y=max(0,min(1,alpha*x+beta))按元素应用于输入张量，得到输出张量，这里alpha=1/6，beta= 0.5

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

HardSwish

按元素应用 HardSwish 函数, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x),这里alpha=1/6, beta=0.5

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

Elu

给定输入张量，函数f(x)=alpha*(exp(x)-1.)forx<0,f(x)=xforx>=0 按元素应用于输入张量，得到输出张量，这里alpha=1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exSwish

按元素应用Sigmoid线性单元函数,Swish函数也称为SiLU函数,y=x*sigmoid(x)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exMish

给定输入张量，函数mish(x)=x*tanh(softplus(x))*=x*tanh(ln(1+e^{x}))按元素应用于输入张量，得 到输出张量

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exGelu

给定输入张量，函数y =x∗Φ(x)按元素应用于输入张量，得到输出张量，当Φ(x)函数设置成tanh时，

$y = 0.5 * x * (1 + Tanh(\sqrt{\frac{2}{\pi}} * (x + 0.044715 * x^3)))$

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

其他支持

- 多核联合:尚不支持

Sin

计算给定输入张量单元的Sin

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:尚不支持

Cos

计算给定输入张量单元的Cos

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:尚不支持

Where

根据mask_tensor获取x_tensor或y_tensor的值，当mask_tensor位置为True时，取x_tensor对 应位置的值，否则取y_tensor的值。

输入列表

- mask_tensor:T1
    - shape:[batch,channel,height,width]
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子
- x_tensor:T2
    - shape:[batch,channel,height,width]

    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子
- y_tensor:T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子

输出列表

- output:T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exConvStreaming

为了减少语音模型的时延，把原始卷积的输入分成若干个时间帧片，将每个时间帧片作为输入，计算卷 积，最后将计算结果缓存起来供后续时间帧使用，从而达到音频流式处理的目的。

输入列表

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width:目前支持width=1。
    - 量化支持
        - per-layer

- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:尚不支持

---

exConvStreamingPRelu/LeakyRelu

exConvStreaming 与 PRelu/LeakyRelu 融合计算, Output = PRelu(exConvStreaming(Input)) 或者 Output= LeakyRelu(exConvStreaming(Input)) 说明及规格限制同exConvStreaming

---

exGlu

门控线性单元(GatedLinearUnit)函数,其中a是输入矩阵的前一半,b是后一半 GLU(a,b)=a⊗σ(b)

输入列表

- Input:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Output:T1
  - shape:[batch,channel/2,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- axis:int(默认值=1)
  - 分割输入的维度
  - 约束:仅支持1

其他支持

- 多核联合:尚不支持

---

exMatMul

矩阵乘法操作

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

输入列表

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]

        - 量化支持
            - per-layer
- B:T1
    - shape:[b,k,1,m]
    - 量化支持
        - per-layer
- C(optional):T1,T2
    - shape:[b,n,1,m]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- Y:T1
    - shape:[b,n,1,m]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)

属性列表

- c_type:strings(默认值“add”)
    - 支持add和mul可选,表示C是做加法还是乘法

其他支持

- 多核联合:不支持

---

exNorm

对输入数据进行各种归一化操作，目前支持LayerNormalization、RMSNormalization和MeanVarianceNormalization三种：

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot W + B$$

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{RMS}^2 + \epsilon}} \cdot W + B$$

$$\text{MeanVarianceNorm}(x) = \frac{x - \mu}{\sqrt{\text{Variance} + \epsilon}} \cdot W + B$$

其中： x是输入向量。 μ和σ²分别是输入向量的均值和方差：

$$\mu = \frac{1}{H} \sum_{i=1}^{H} x_i$$

$$\sigma^2 = \frac{1}{H} \sum_{i=1}^{H} (x_i - \mu)^2$$

RMS是输入向量均方根：

$$\text{RMS} = \sqrt{\frac{1}{H} \sum_{i=1}^{H} x_i^2}$$

Variance是输入向量的等效方差：

$$\text{Variance} = \frac{1}{H} \sum_{i=1}^{H} x_i^2 - \mu^2$$

ϵ是一个很小的数，用于防止除零操作。 W 和B是可选择的参数。

输入列表

- X:T1
  - shape:[batch,channel,height,width]
- W(optional):T1
  - shape:[batch,channel,height,width]
- B(optional):T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1

    - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

属性列表

- epsilon:float(默认值1e-05)
    - 用来防止除0的epsilon数值
- mode:string
    - 支持layer、rms和mean_var三种Normalization模式可选
- axes:int64[](默认值[1])
    - 归一化轴，目前只支持[1]和[1,2,3]

其他支持

- 多核联合:尚不支持

---

exSDPAttention

计算查询(query)、键(key)和值(value)之间的缩放点积注意力(SDPA)

输入列表

- query:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
    - 量化支持
        - per-layer
- key:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- value:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
    - 量化支持
        - per-layer
- mask(optional):T1,T2

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 默认支持任意维度广播操作
        - 开启FA支持后,只支持[b,c,1,1],[1,c,1,1]和[1,c,h,w]三种规格
- scale(optional):T3

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

其他支持

- 多核联合:RK3576支持/RV1126B不支持
- FlashAttentionV2(FA):支持
    - 基于https://arxiv.org/abs/2307.08691实现,通过高速缓存内循环实现加速以及减少宽带使 用，但是会导致模型增大，请根据具体场景和模型选择是否开启使用
    - RKNPU驱动版本>=0.9.7
    - RKNPURuntime库(librknnrt.so)版本>=2.1.0
    - 支持数据类型:float16

---

exWindow

exWindow来源于swin_transformer的window_partition和window_reverse操作。 原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size× window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。 pytorch实现的计算方法如下：

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

输入列表

- X:T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor),float16(Tensor)

属性列表

- mode:string
  - 支持模式:partition,reverse,partition_num_first
    - partition:参考window_partition实现;
    - reverse:参考window_reverse实现;
    - partition_num_first: 与 partition 的区别在于，partition_num_first 的 H,W 按照num_windows进行划分，而partition的H,W按照window_sizes进行划分。

- window_sizes:listofints
    - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是 基于这个大小。
- num_windows:listofints
    - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。

其他支持

- 多核联合:尚不支持

RK2118

Add

执行元素级二进制加法(C=A+B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

AddRelu

Add与Relu融合计算,C=Relu(A+B)

输入列表

- 同Add

输出列表

- 同Add

BatchNormalization

按照论文https://arxiv.org/abs/1502.03167中的描述对输入张量进行批量归一化计算

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)

属性列表

- epsilon:float（默认值=1e-5）
    - 除以标准差时加上防止除0的实数
    - epsilon∈(0,∞]
- momentum:float

- 训练时的滑动平均参数

Concat

将一系列向量在axis指定的方向上组合成一个向量，所有向量除axis指定的方向外大小必须相同

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T:float16(Tensor)

属性列表

- axisint64
  - axis∈{0,1,2,3}

Convolution

卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1

    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - 量化支持
        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

数据类型约束

- T1:float16(Tensor)

Depthwise Convolution

深度可分离卷积

输入列表

- input_tensor:T1
    - shape:[batch,channel,height,width]

        - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
    - 量化支持
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,8]
        - kernel_w∈[1,8]
    - 量化支持
        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

数据类型约束

- T1:float16(Tensor)

ConvolutionRelu

Convolution与Relu融合计算,Output=Relu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionClip

Convolution与Clip融合计算,Output=Clip(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution 与 PRelu/LeakyRelu 融合计算, Output = PRelu(Conv(Input)) 或者 Output = LeakyRelu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionAdd

Convolution与Add融合计算,Output=Add(Conv(Input0),Input1) 说明及规格限制同Convolution

---

ConvolutionSigmoid

Convolution与Sigmoid融合计算,Output=Sigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionTanh

Convolution与Tanh融合计算,Output=Tanh(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionSoftplus

Convolution与Softplus融合计算,Output=Softplus(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSigmoid

Convolution与HardSigmoid融合计算,Output=HardSigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSwish

Convolution与HardSwish融合计算,Output=HardSwish(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionElu

Convolution与Elu融合计算,Output=Elu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionSwish

Convolution与Swish融合计算,Output=Swish(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionMish

Convolution与Mish融合计算,Output=Mish(Conv(Input)) 说明及规格限制同Convolution

---

ConvTranspose

转置卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
- 量化支持
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)

    - stride_h∈[1,8]
    - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) 设置pad时注意: 不支持 kernel_h*dilations_h-dilations_h-pads_top<0 不支持kernel_w*dilations_w-dilations_w-pads_left<0 不支持stride_h*(height-1)-pads_top+1<output_h 不支持stride_w*(width-1)-pads_left+1<output_w
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group:仅支持1,当且仅当num_input=num_output时支持num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - 当dilation不等于(1,1)时，只支持kernel=(1,1)

数据类型约束

- T1:float16(Tensor)

Div

执行元素级二进制除法(C=A/B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - [b,c,1,1]
        - [scalar]
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持

        - [b,c,1,1]
        - [scalar]

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

根据给定的 shape 和广播规则广播输入张量。广播规则类似于 numpy.array(input) * numpy.ones(shape)：维度右对齐；两个对应维度必须具有相同的值，或者其中一个等于1

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- shape:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:int64(Tensor)

广播约束

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

LSTM

长短期记忆网络（LSTM，Long Short-Term Memory） LSTM扩展以及变体命名为exLSTM算子，参数项中指明 （extern）的项为exLSTM独有的参数项。 计算公式如下：
Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

输入列表

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192],大于1时仅支持4的倍数
    - inputsize∈(0,8192]
  - 量化支持
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - 量化支持
    - per-layer

        - per-channel
- B:T2
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - 量化支持
        - per-layer
        - per-channel
- sequence_lens:T3(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192],大于1时仅支持4的倍数
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

输出列表

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持

        - per-layer

- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192],大于1时仅支持4的倍数
        - hiddensize∈(0,8192]
    - 量化支持
        - per-layer

属性列表

- direction(extern):string
    - 指定LSTM的运算方向
    - forward:指定LSTM的运算方向为前向
    - reverse:指定LSTM的运算方向为反向
    - bidirectional:指定LSTM的运算方向为双向
- sequence_size(extern):int
    - 指定LSTM输入的seqsize，无限制，建议4对齐
- hidden_size(extern):int
    - LSTM单元中的hiddensize，无限制，建议8对齐
- proj_size(extern):int
    - projection时的proj_size，projsize∈[0,hiddensize]
    - 目前限定0，即尚不支持projection功能
- input_forget(extern):int
    - cifg变种的选择:1(T)or0(F)目前限定0，即尚不支持
- has_projection(extern):int
    - projection变种:1(T)or0(F)目前限定0，即尚不支持
- input_layout(extern):int
    - 指定与对应输入shape含义一致的layout

    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size。
    - snc：指定layout对应的输入shape为[seqs,batches,input_size]
    - (sn)c：指定layout对应的输入shape为[seqs*batches,input_size,1,1]
- output_layout(extern):int
    - 指定与对应输出shape含义一致的layout
    - 要求填写指定的layout，同时要求填写该op实际对应的batch_size、sequence_size、hidden_size、directions。
    - sbnc：指定layout对应的输出shape为[seqs,directions,batches,hidden_size]
    - (sn)c：指定layout对应的输出shape为[seqsbatches,directionsinput_size,1,1]

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

其他支持

- 多核联合:暂不支持

Max

计算输入张量的元素级最大值

输入列表

- A:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer/per-channel

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:暂不支持

Mul

执行元素级二进制乘法(C=A*B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持

    - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

MulRelu

Mul与Relu融合计算,C=Relu(A*B)

输入列表

- 同Mul

输出列表

- 同Mul

其他支持

- 多核联合:支持

Pad

给定一个包含要填充的数据的张量(data),包含轴的开始和结束填充数值的张量(pads),模式(mode),常 量数值(constant_value),生成一个填充张量(output)

输入列表

- data:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- pads:T2
    - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
    - shape:[1]
    - 量化支持
        - per-layer

输出列表

- output:T1

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

属性列表

- mode:string(默认值constant)
    - 支持模式:constant,reflect
        - constant无限制
        - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

其他支持

- 多核联合:尚不支持

MaxPool

MaxPool消耗输入张量X并根据内核大小、步幅大小和pad 长度在张量上应用最大池化。最大池化包括根据内核大小计算输入张量子集的所有值的最大值，并将数 据下采样到输出张量Y 中以进行进一步处理。输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads 的情况下）或使用自动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

输出列表

- output:T1
    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)

    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
    - 注,不支持kernel_h=1且kernel_w=1
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool使用输入张量X并对同一通道中的值应用最大池化。这相当于MaxPool的kernel_shape 大小等于input的空间维度。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

输出列表

- output:T1

    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7] 注,不支持kernel_h=1且kernel_w=1
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

- 不支持(工具端转换成卷积实现)

GlobalAveragePool

- 不支持(工具端转换成卷积实现)

Pow

Z =XY 指数运算，采用输入数据（张量）和指数，并产生一个输出数据（张量）

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- Y:T2
  - shape:[1]
    - 当前仅支持0、1、2、3、0.5、-0.5

输出列表

- Z:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

其他支持

- 多核联合:尚不支持

Relu

将输入张量的负值设为零，正值保持不变

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 广播支持
    - 无

输出列表

- output:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:暂不支持

Clip/ReLU6

将输入张量的值裁剪到[0,6]范围内

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:暂不支持

PRelu

将输入张量的负值按可学习参数缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 广播支持
        - 无

输出列表

- output:T1

    - shape:[batch,channel,height,width]

属性列表

- slope:PRelu系数
    - 仅支持单个标量或C维度系数

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:暂不支持

LeakyRelu

将输入张量的负值按固定比例缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:暂不支持

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

输入列表

- input:T1
    - shape:[batch,channel,height,width]

    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)

属性列表

- shape:int
    - 指定输出张量的形状

其他支持

- 多核联合:尚不支持

Resize

调整输入张量的大小。一般来说，它将输出张量中的每个值计算为输入张量中邻域（即采样位置）的加 权平均值

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- roi(optional):T2
    - 目前暂不支持
- scales(optional):T2
    - 沿每个维度的缩放比例数组
- sizes(optional):T3
    - 输出张量的目标大小

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

属性列表

- antialias:int（默认值为0）
  - 目前不支持设置
- axes:int[]
  - 目前不支持设置
- coordinate_transformation_mode:strings（默认值为”half_pixel”）
  - 目前仅支持half_pixel，pytorch_half_pixel，align_corners三种
- cubic_coeff_a:float（默认为-0.75）
  - 目前不支持设置
- exclude_outside:int（默认值为0）
  - 目前不支持设置
- extrapolation_value:float（默认为0）
  - 目前不支持设置
- keep_aspect_ratio_policy:strings（默认值为”stretch”）
  - 目前不支持设置
- mode:strings（默认值为”nearest”）
  - 目前仅支持nearest和linear两种可配置
- nearest_mode:strings（默认值为”round_prefer_floor”）
  - 目前不支持设置 支持说明 目前仅支持宽高方向的整数倍缩放,且缩放倍数为2-8的整数倍,如果是(1,c,1,1)则支持放大到(1,c,h, w)

Slice

获得当前向量的一个切片，具体规则和numpy切片相同

输入列表

- input_tensor:T

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T:float16(Tensor)

属性列表

- starts:int(Tensor)
    - 切分的起始位置:无限制
- ends:int(Tensor)
    - 切分的终止位置:无限制
- axes:int(Tensor)
    - 选取切分的轴:支持任意0~3轴，支持同时多轴选择
- steps:int(Tensor)
    - 选取切分对应轴的步长
    - 当height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]%subc==0&&steps[1]<=subc时,steps可以不为1，否则只有steps==[1,1,1,1]时，才可 通过NPU运行SliceOP

Softmax

该运算符计算给定输入的归一化指数值：

`Softmax(input, axis) = exp(input – max(input, axis)) / Σ_axis exp(input – max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]

        - width: · axis=1,无限制; · axis=3/-1,width∈(0,8192] 且受限于transpose的规格限制

输出列表

- output:T1
    - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

属性列表

- axis:int64
    - 做softmax的轴:1,3，即channel和width方向

---

exSoftmax13

该运算符计算给定输入的归一化指数值：

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - width: · axis=1,无限制; · axis=3/-1,width∈(0,8192] 且受限于transpose的规格限制

输出列表

- output:T1
    - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

属性列表

- axis:int64
    - 做softmax的轴:1,3，即channel和width方向

---

exSoftmaxMask

- 尚不支持

Split

将向量沿着axis方向分成num_outputs个向量，split用于指定切分后向量在axis方向上的大小

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - 一或多个输出向量，由num_outputs属性决定
  - shape:根据切分情况决定
  - 量化支持
    - per-layer

数据类型约束

- T:float16(Tensor)

属性列表

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- 执行元素级二进制减法(C=A-B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer
    - 广播支持
        - [b,c,1,1]
        - [scalar]
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - [b,c,1,1]
        - [scalar]

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

Tile

通过平铺给定的张量构造张量,这与Numpy中的函数tile相同，但没有广播

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

对输入张量进行转置

输入列表

- X:T1
    - shape:[n1,c1,h1,w1]
        - 详细约束规则见属性列表
    - 量化支持
        - per-layer

输出列表

- Y:T1
    - shape:[n2,c2,h2,w2]
        - 详细约束规则见属性列表
    - 量化支持
        - per-layer

属性列表

- perm转置的轴顺序: 不同perm参数限制如下：(其中p=268435456,q=65536)
    - perm=[0,1,2,3]
        - 无限制
    - perm=[0,1,3,2]
        - T1:h1%8=0,h1*w1*<8p,h1*w1*<2q
    - perm=[0,2,3,1]
        - T1:c1<8192,h1*w1%8=0，w1*c1<p*8
    - perm=[0,2,1,3]
        - T1:h1%8=0,h1*w1*<8p,h1*w1*<2q,c1*w1<p

- perm=[0,3,1,2]
  - T1:h1*w1<p,w1<8192,h1*c1<p
- perm=[0,3,2,1]
  - T1:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1<p*8
- perm=[1,0,2,3]
  - T1:h1*w1<8*p,n1<8192,h1*w1<p
- perm=[1,0,3,2]
  - T1:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
- perm=[1,2,0,3]
  - T1:n1*w1<p*8,c1*h1*w1p,w1%8=0
- perm=[1,2,3,0]
  - T1:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
  - T1:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
  - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
  - T1:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
  - T1:c1*h1*w1<p,n1*w1<p8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
  - T1:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
  - T1:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
  - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
  - T1:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%8=0
- perm=[3,0,1,2]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%8=0
- perm=[3,2,0,1]
  - T1:h1*w1<p,n1*c1<8192,n1*c1*w1<p
- perm=[3,2,1,0]

        - T1:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*8,h1%8=0
    - perm=[3,1,0,2]
        - T1:h1*w1*<p,c1*w1<8192,n1*h1<p
    - perm=[3,1,2,0]
        - T1:h1*w1*<p,n1*c1<8192,n1*h1*<p,n1*c1*w1<p

数据类型约束

- T1:int16(Tensor),float16(Tensor)

其他支持

- 多核联合:暂不支持

Sigmoid

给定输入张量，函数y=1/(1+exp(-x))按元素应用于输入张量，得到输出张量

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)

Tanh

计算给定输入张量单元的双曲正切

输入列表

- X:T1
    - shape:[batch,channel,height,width]

    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:float16(Tensor)

Softplus

按元素应用Softplus函数,y=ln(exp(x)+1)

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:float16(Tensor)

HardSigmoid

给定输入张量，函数y=max(0,min(1,alpha*x+beta))按元素应用于输入张量，得到输出张量，这里alpha=1/6，beta=0.5

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)

HardSwish

按元素应用 HardSwish 函数, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x),这里alpha=1/6,beta=0.5

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:float16(Tensor)

Elu

给定输入张量，函数f(x)=alpha*(exp(x)-1.)forx<0,f(x)=xforx>=0按元素应用于输入张量，得到 输出张量，这里alpha=1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)

---

exSwish

按元素应用Sigmoid线性单元函数,Swish函数也称为SiLU函数,y=x*sigmoid(x)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:float16(Tensor)

---

exMish

给定输入张量，函数mish(x)=x*tanh(softplus(x))*=xtanh(ln(1+e^{x}))按元素应用于输入张量，得 到输出张量

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1

- shape:[batch,channel,height,width]
- 量化支持
  - per-layer

数据类型约束

- T1:float16(Tensor)

---

exGelu

给定输入张量，函数y =qx∗Φ(x)按元素应用于输入张量，得到输出张量，当Φ(x)函数设置成tanh时， y =0.5∗x∗(1+Tanh(2 ∗(x+0.044715∗x3))) π

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)

Where

根据mask_tensor获取x_tensor或y_tensor的值，当mask_tensor位置为True时，取x_tensor对 应位置的值，否则取y_tensor的值。

输入列表

- mask_tensor:T1
  - shape:[batch,channel,height,width]
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束

        - 广播约束同Expand算子
- x_tensor:*T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子
- y_tensor:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
    - 广播约束
        - 广播约束同Expand算子

输出列表

- output:T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:bool(Tensor)
- T2:float16(Tensor)

---

exConvStreaming

为了减少语音模型的时延，把原始卷积的输入分成若干个时间帧片，将每个时间帧片作为输入，计算卷 积，最后将计算结果缓存起来供后续时间帧使用，从而达到音频流式处理的目的。

输入列表

- input_tensor:T1
    - shape:[batch,channel,height,width]

        - width:目前支持width=1。
    - 量化支持
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - 量化支持
        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

数据类型约束

- T1:float16(Tensor)

其他支持

- 多核联合:尚不支持

exConvStreamingPRelu/LeakyRelu

exConvStreaming 与 PRelu/LeakyRelu 融合计算, Output = PRelu(exConvStreaming(Input)) 或者 Output= LeakyRelu(exConvStreaming(Input)) 说明及规格限制同exConvStreaming

---

exGlu

门控线性单元(GatedLinearUnit)函数,其中a是输入矩阵的前一半,b是后一半 GLU(a,b)=a⊗σ(b)

输入列表

- Input:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Output:T1
  - shape:[batch,channel/2,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:float16(Tensor)

属性列表

- axis:int(默认值=1)
  - 分割输入的维度
  - 约束:仅支持1

---

exMatMul

矩阵乘法操作

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

输入列表

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]

        - 量化支持
            - per-layer
- B:T1
    - shape:[b,k,1,m]
    - 量化支持
        - per-layer
- C(optional):T1,T2
    - shape:[b,n,1,m]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- Y:T1
    - shape:[b,n,1,m]
    - 量化支持
        - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(Tensor)

属性列表

- c_type:strings(默认值“add”)
    - 支持add和mul可选,表示C是做加法还是乘法

---

exWindow

exWindow来源于swin_transformer的window_partition和window_reverse操作。 原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size× window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。 pytorch实现的计算方法如下：

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

输入列表

- X:T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:float16(Tensor)

属性列表

- mode:string
  - 支持模式:partition,reverse,partition_num_first
    - partition:参考window_partition实现;
    - reverse:参考window_reverse实现;
    - partition_num_first: 与 partition 的区别在于，partition_num_first 的 H,W 按照num_windows进行划分，而partition的H,W按照window_sizes进行划分。

- window_sizes:listofints
    - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是 基于这个大小。
- num_windows:listofints
    - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。

其他支持

- 多核联合:尚不支持

RV1103B/RV1106B

Add

执行元素级二进制加法(C=A+B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(constTensor)

AddRelu

Add与Relu融合计算,C=Relu(A+B)

输入列表

- 同Add

输出列表

- 同Add

其他支持

- 多核联合:支持

BatchNormalization

按照论文https://arxiv.org/abs/1502.03167中的描述对输入张量进行批量归一化计算

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)

属性列表

- epsilon:float（默认值=1e-5）

    - 除以标准差时加上防止除0的实数
    - epsilon∈(0,∞]
- momentum:float
    - 训练时的滑动平均参数

Concat

将一系列向量在axis指定的方向上组合成一个向量，所有向量除axis指定的方向外大小必须相同

输入列表

- input_tensor:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T:int8(Tensor)

属性列表

- axisint64
    - axis∈{0,1,2,3}

Convolution

卷积

输入列表

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。

    - 量化支持
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - 量化支持
        - per-layer
        - per-channel

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

属性列表

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

数据类型约束

- T1:int8(Tensor)

Depthwise Convolution

深度可分离卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
  - 量化支持
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

数据类型约束

- T1:int8(Tensor)

ConvolutionRelu

Convolution与Relu融合计算,Output=Relu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionClip

Convolution与Clip融合计算,Output=Clip(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution 与 PRelu/LeakyRelu 融合计算, Output = PRelu(Conv(Input)) 或者 Output = LeakyRelu(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionAdd

Convolution与Add融合计算,Output=Add(Conv(Input0),Input1) 说明及规格限制同Convolution

---

ConvolutionSigmoid

Convolution与Sigmoid融合计算,Output=Sigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionTanh

Convolution与Tanh融合计算,Output=Tanh(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionSoftplus

Convolution与Softplus融合计算,Output=Softplus(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSigmoid

Convolution与HardSigmoid融合计算,Output=HardSigmoid(Conv(Input)) 说明及规格限制同Convolution

---

ConvolutionHardSwish

Convolution与HardSwish融合计算,Output=HardSwish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionElu

Convolution与Elu融合计算,Output=Elu(Conv(Input)) 说明及规格限制同Convolution

ConvolutionSwish

Convolution与Swish融合计算,Output=Swish(Conv(Input)) 说明及规格限制同Convolution

ConvolutionMish

Convolution与Mish融合计算,Output=Mish(Conv(Input)) 说明及规格限制同Convolution

ConvTranspose

转置卷积

输入列表

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: 当dilated_kernel_h>1时,width<16383。此外,对首层输入width存在限制，详见模型输入说明。
- 量化支持
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - 量化支持
    - per-layer
    - per-channel

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

属性列表

- strides:int(strides_h,strides_w)
  - stride_h∈[1,8]
  - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) 设置pad时注意: 不支持kernel_h*dilations_h-dilations_h-pads_top<0 不支持kernel_w*dilations_w-dilations_w-pads_left<0 不支持stride_h*(height-1)-pads_top+1<output_h 不支持stride_w*(width-1)-pads_left+1<output_w
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
  - group:仅支持1,当且仅当num_input=num_output时支持num_output
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]
  - 当dilation不等于(1,1)时，只支持kernel=(1,1)

数据类型约束

- T1:int8(Tensor)

ConvTranposeRelu

ConvTranpose与Relu融合计算,Output=Relu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposeClip

ConvTranpose与Clip融合计算,Output=Clip(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranpose与PRelu/LeakyRelu融合计算,Output=PRelu(ConvTranpose(Input))或者Output=LeakyRelu(ConvTranpose(Input)) 说明及规格限制同ConvTranspose

---

Div

执行元素级二进制除法(C=A/B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - [b,c,1,1]
    - [scalar]

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

根据给定的 shape 和广播规则广播输入张量。广播规则类似于 numpy.array(input) * numpy.ones(shape)：维度右对齐；两个对应维度必须具有相同的值，或者其中一个等于1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

- shape:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:int64(Tensor)

广播约束

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

Max

计算输入张量的元素级最大值

输入列表

- A:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持

        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

数据类型约束

- T1:int8(Tensor)

其他支持

- 多核联合:暂不支持

Mul

执行元素级二进制乘法(C=A*B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - 支持任意维度广播操作
        - 广播约束同Expand算子

输出列表

- C:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(constTensor)

MulRelu

Mul与Relu融合计算,C=Relu(A*B)

输入列表

- 同Mul

输出列表

- 同Mul

其他支持

- 多核联合:支持

Pad

给定一个包含要填充的数据的张量(data),包含轴的开始和结束填充数值的张量(pads),模式(mode),常 量数值(constant_value),生成一个填充张量(output)

输入列表

- data:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - 量化支持

        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

属性列表

- mode:string(默认值constant)
    - 支持模式:constant,reflect
        - constant无限制
        - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

其他支持

- 多核联合:尚不支持

MaxPool

MaxPool消耗输入张量X并根据内核大小、步幅大小和pad长度在张量上应用最大池化。最大池化包括 根据内核大小计算输入张量子集的所有值的最大值，并将数据下采样到输出张量Y中以进行进一步处理。 输出空间形状的计算方式有所不同，具体取决于是否使用显式填充（在使用pads的情况下）或使用自 动填充（在使用auto_pad的情况下）。仅使用显式填充

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

输出列表

- output:T1

    - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - pad的方式:仅支持NOTSET
- ceil_mode:int64
    - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool使用输入张量X并对同一通道中的值应用最大池化。这相当于MaxPool的kernel_shape 大小等于input的空间维度。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

输出列表

- output:T1
  - shape:[batch,channel,height,width]

属性列表

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - pad的方式:仅支持NOTSET
- ceil_mode:int64
  - 使用ceil或floor的方式计算输出的shape:不支持
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad是否包含pad数值进行计算:1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

数据类型约束

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

- 不支持(工具端转换成卷积实现)

GlobalAveragePool

- 不支持(工具端转换成卷积实现)

Relu

将输入张量的负值设为零，正值保持不变

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 无

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)

其他支持

- 多核联合:暂不支持

Clip/ReLU6

将输入张量的值裁剪到[0,6]范围内

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 无

输出列表

- output:T1

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)

其他支持

- 多核联合:暂不支持

PRelu

将输入张量的负值按可学习参数缩放，正值保持不变

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel
    - 广播支持
        - 无

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer/per-channel

属性列表

- slope:PRelu系数
    - 仅支持单个标量或C维度系数

数据类型约束

- T1:int8(Tensor)

其他支持

- 多核联合:暂不支持

LeakyRelu

将输入张量的负值按固定比例缩放，正值保持不变

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 无

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)

其他支持

- 多核联合:暂不支持

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持

        - per-layer

数据类型约束

- T1:int8(Tensor)

属性列表

- shape:int
    - 指定输出张量的形状

其他支持

- 多核联合:尚不支持

Resize

调整输入张量的大小。一般来说，它将输出张量中的每个值计算为输入张量中邻域（即采样位置）的加 权平均值

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- roi(optional):T2
    - 目前暂不支持
- scales(optional):T2
    - 沿每个维度的缩放比例数组
- sizes(optional):T3
    - 输出张量的目标大小

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)

- T2:float(Tensor)
- T3:int64(Tensor)

属性列表

- antialias:int（默认值为0）
  - 目前不支持设置
- axes:int[]
  - 目前不支持设置
- coordinate_transformation_mode:strings（默认值为”half_pixel”）
  - 目前仅支持half_pixel，pytorch_half_pixel，align_corners三种
- cubic_coeff_a:float（默认为-0.75）
  - 目前不支持设置
- exclude_outside:int（默认值为0）
  - 目前不支持设置
- extrapolation_value:float（默认为0）
  - 目前不支持设置
- keep_aspect_ratio_policy:strings（默认值为”stretch”）
  - 目前不支持设置
- mode:strings（默认值为”nearest”）
  - 目前仅支持nearest和linear两种可配置
- nearest_mode:strings（默认值为”round_prefer_floor”）
  - 目前不支持设置 支持说明 目前仅支持宽高方向的整数倍缩放,且缩放倍数为2-8的整数倍,如果是(1,c,1,1)则支持放大到(1,c,h, w)

Slice

获得当前向量的一个切片，具体规则和numpy切片相同

输入列表

- input_tensor:T
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T
  - shape:[batch,channel,height,width]

    - 量化支持
        - per-layer

数据类型约束

- T:int8(Tensor)

属性列表

- starts:int(Tensor)
    - 切分的起始位置:无限制
- ends:int(Tensor)
    - 切分的终止位置:无限制
- axes:int(Tensor)
    - 选取切分的轴:支持任意0~3轴，支持同时多轴选择
- steps:int(Tensor)
    - 选取切分对应轴的步长
    - 当height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]%subc==0&&steps[1]<=subc时,steps可以不为1，否则只有steps==[1,1,1,1]时，才可 通过NPU运行SliceOP

Softmax

该运算符计算给定输入的归一化指数值：

`Softmax(input, axis) = exp(input – max(input, axis)) / Σ_axis exp(input – max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - 受限于transpose的规格限制

输出列表

- output:T1
    - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor)

属性列表

- axis:int64
    - 做softmax的轴:1,3，即channel和width方向

---

exSoftmax13

该运算符计算给定输入的归一化指数值：

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

“axis”属性表示Softmax执行的维度。输出张量具有相同的形状并包含相应输入的Softmax值。

输入列表

- input:T1
    - shape:[batch,channel,height,width]
        - 受限于transpose的规格限制

输出列表

- output:T1
    - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor)

属性列表

- axis:int64
    - 做softmax的轴:1,3，即channel和width方向

---

exSoftmaxMask

- 尚不支持

Split

将向量沿着axis方向分成num_outputs个向量，split用于指定切分后向量在axis方向上的大小

输入列表

- input_tensor:T

    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T
    - 一或多个输出向量，由num_outputs属性决定
    - shape:根据切分情况决定
    - 量化支持
        - per-layer

数据类型约束

- T:int8(Tensor)

属性列表

- axis:int
    - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- 执行元素级二进制减法(C=A-B),支持多向Numpy样式的广播

输入列表

- A:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
    - 广播支持
        - [b,c,1,1]
        - [scalar]
- B:T1,T2
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

    - 广播支持
        - [b,c,1,1]
        - [scalar]

输出列表

- C:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(constTensor)

Tile

通过平铺给定的张量构造张量,这与Numpy中的函数tile相同，但没有广播

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

对输入张量进行转置

输入列表

- X:T1
  - shape:[n1,c1,h1,w1]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

输出列表

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - 详细约束规则见属性列表
  - 量化支持
    - per-layer

属性列表

- perm转置的轴顺序: 不同perm参数限制如下：(其中p=268435456,q=65536)
  - perm=[0,1,2,3]
    - 无限制
  - perm=[0,1,3,2]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q
  - perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
    - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
  - perm=[0,2,1,3]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1*w1<p
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1*w1<p
  - perm=[0,3,1,2]
    - T1,T2:h1*w1<p,w1<8192,h1*c1<p
  - perm=[0,3,2,1]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1<p*16
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1<p*8

- perm=[1,0,2,3]
  - T1:h1*w1<16*p,n1<8192,h1*w1<p
  - T2:h1*w1<8*p,n1<8192,h1*w1<p
- perm=[1,0,3,2]
  - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
  - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
- perm=[1,2,0,3]
  - T1:n1*w1<p*16,c1*h1*w1<p,w1%16=0
  - T2:n1*w1<p*8,c1*h1*w1p,w1%8=0
- perm=[1,2,3,0]
  - T1:h1*w1<p,n1<p*16,c1*h1*w1<p*16,n1*w1<p*16
  - T2:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
  - T1:c1*w1<8192,n1*h1<p,w1%16=0
  - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
  - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
  - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
  - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
  - T1:c1*h1*w1<p,n1*w1<p*16,c1<8192,n1*w1%4=0
  - T2:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
  - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
  - T1:n1*c1*h1*w1<p,h1*w1%16=0
  - T2:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
  - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%16=0
  - T2:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
  - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%16=0

        - T2:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*w1%8=0
    - perm=[3,0,1,2]
        - T1:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*h1*w1%16=0
        - T2:h1*w1*<p,n1*c1<8192,c1*h1*<p,n1*w1<8192,n1*h1*w1%8=0
    - perm=[3,2,0,1]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*c1*w1<p
    - perm=[3,2,1,0]
        - T1:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*16,h1%16=0
        - T2:c1*h1*w1<p,h1*w1*<8192,2*h1*w1*<q,n1*c1<p*8,h1%8=0
    - perm=[3,1,0,2]
        - T1,T2:h1*w1*<p,c1*w1<8192,n1*h1<p
    - perm=[3,1,2,0]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*h1*<p,n1*c1*w1<p

数据类型约束

- T1:int8(Tensor)
- T2:int16(Tensor)

其他支持

- 多核联合:暂不支持

Sigmoid

给定输入张量，函数y=1/(1+exp(-x))按元素应用于输入张量，得到输出张量

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)

Tanh

计算给定输入张量单元的双曲正切

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor)

Softplus

按元素应用Softplus函数,y=ln(exp(x)+1)

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor)

HardSigmoid

给定输入张量，函数y=max(0,min(1,alpha*x+beta))按元素应用于输入张量，得到输出张量，这里alpha=1/6，beta= 0.5

输入列表

- input:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

输出列表

- output:T1
    - shape:[batch,channel,height,width]
    - 量化支持
        - per-layer

数据类型约束

- T1:int8(Tensor)

HardSwish

按元素应用 HardSwish 函数, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x),这里alpha=1/6, beta=0.5

输入列表

- X:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

输出列表

- Y:T1
    - shape:[batch,channel,height,width]
    - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor)

Elu

给定输入张量，函数f(x)=alpha*(exp(x)-1.)forx<0,f(x)=xforx>=0 按元素应用于输入张量，得到输出张量，这里alpha=1

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)

---

exSwish

按元素应用Sigmoid线性单元函数,Swish函数也称为SiLU函数,y=x*sigmoid(x)

输入列表

- X:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Y:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor)

---

exMish

给定输入张量，函数mish(x)=x*tanh(softplus(x))*=xtanh(ln(1+e^{x}))按元素应用于输入张量，得 到输出张量

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)

---

exGelu

给定输入张量，函数y =x∗Φ(x)按元素应用于输入张量，得到输出张量，当Φ(x)函数设置成tanh时，

$y = 0.5 * x * (1 + Tanh(\sqrt{\frac{2}{\pi}} * (x + 0.044715 * x^3)))$

输入列表

- input:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

输出列表

- output:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)

Where

根据mask_tensor获取x_tensor或y_tensor的值，当mask_tensor位置为True时，取x_tensor对 应位置的值，否则取y_tensor的值。

输入列表

- mask_tensor:T1
  - shape:[batch,channel,height,width]
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束
    - 广播约束同Expand算子
- x_tensor:*T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束
    - 广播约束同Expand算子
- y_tensor:T1
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
  - 广播约束
    - 广播约束同Expand算子

输出列表

- output:T2
  - shape:[batch,channel,height,width]
  - 量化支持
    - per-layer

数据类型约束

- T1:bool(Tensor)
- T2:int8(Tensor)

---

exGlu

门控线性单元(GatedLinearUnit)函数,其中a是输入矩阵的前一半,b是后一半 GLU(a,b)=a⊗σ(b)

输入列表

- Input:T1
  - shape:[batch,channel,height,width]
  - 量化支持:per-layer

输出列表

- Output:T1
  - shape:[batch,channel/2,height,width]
  - 量化支持:per-layer

数据类型约束

- T1:int8(Tensor)

属性列表

- axis:int(默认值=1)
  - 分割输入的维度
  - 约束:仅支持1

---

exMatMul

矩阵乘法操作

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

输入列表

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - 量化支持
    - per-layer

- B:T1
  - shape:[b,k,1,m]
  - 量化支持
    - per-layer
- C(optional):T1,T2
  - shape:[b,n,1,m]
  - 量化支持
    - per-layer
  - 广播支持
    - 支持任意维度广播操作
    - 广播约束同Expand算子

输出列表

- Y:T1
  - shape:[b,n,1,m]
  - 量化支持
    - per-layer

数据类型约束

- T1:int8(Tensor)
- T2:float(Tensor)

属性列表

- c_type:strings(默认值“add”)
  - 支持add和mul可选,表示C是做加法还是乘法

---

exWindow

exWindow来源于swin_transformer的window_partition和window_reverse操作。 原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size× window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。 pytorch实现的计算方法如下：

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

输入列表

- X:T1
  - shape:[batch,channel,height,width]

输出列表

- Y:T1
  - shape:[batch,channel,height,width]

数据类型约束

- T1:int8(Tensor)

属性列表

- mode:string
  - 支持模式:partition,reverse,partition_num_first
    - partition:参考window_partition实现;
    - reverse:参考window_reverse实现;
    - partition_num_first: 与 partition 的区别在于，partition_num_first 的 H,W 按照num_windows进行划分，而partition的H,W按照window_sizes进行划分。

- window_sizes:listofints
    - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是 基于这个大小。
- num_windows:listofints
    - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。

其他支持

- 多核联合:尚不支持
- 3566/3568OP规格介绍
- 3588OP规格介绍
- 3562OP规格介绍
- 1103/1106OP规格介绍
- 3576/RV1126BOP规格介绍
- 2118OP规格介绍
- 1103B/1106BOP规格介绍

CPU OPs 规格介绍

注:

- RK2118不支持所有CPU 算子
- RV1106/RV1103/RV1106B/RV1103B仅支持部分CPU 算子如下
    - Concat, DataConvert, Div, exLayerNorm, Exp, Gather, ReduceL2, ReduceMax, Reshape, ScatterND,Softmax,Pow

---

Add

- **描述**：加法操作
- **规格约束**：无限制

AveragePool

- **描述**：平均池化
- **规格约束**：无限制

ArgMin

- **描述**：取最小值的index
- **规格约束**：无限制

ArgMax

- **描述**：取最大值的index
- **规格约束**：无限制

BatchNormalization

- **描述**：批量归一化
- **规格约束**：无限制

Cast

- **描述**：数据类型转换
- **规格约束**： SRC支持:float32/bool/int8/float16/int32/int64 DST支持:float32/int8/int32/float16

Clip

- **描述**：数据截断激活层
- **规格约束**：无限制

Concat

- **描述**：合并操作
- **规格约束**：axis仅支持{0,1,2,3}

Convolution

- **描述**：卷积操作
- **规格约束**：无限制

ConvTranspose/Deconvolution

- **描述**：转置卷积
- **规格约束**：无限制

ConvTransposePad

- **描述**：组合OP
- **规格约束**：无限制

Cos

- **描述**：余弦函数
- **规格约束**：无限制

DataConvert

- **描述**：数据类型转换
- **规格约束**：仅支持bool/int8/float类型转换

DepthToSpace

- **描述**：通道方向空间方向转换

- **规格约束**：无限制

Div

- **描述**：除法操作
- **规格约束**：无限制

Equal

- **描述**：等于
- **规格约束**：无限制

Exp

- **描述**：指数函数
- **规格约束**：无限制

Flatten

- **描述**：拉平操作
- **规格约束**：无限制

Gather

- **描述**：聚集操作
- **规格约束**：无限制

Greater

- **描述**：大于
- **规格约束**：无限制

GreaterOrEqual

- **描述**：大等于
- **规格约束**：无限制

GRU

- **描述**：门控循环单元
- **规格约束**：无限制

---

exGRU

- **描述**：门控循环单元
- **规格约束**：无限制
    - 说明:ONNX扩展算子

---

exHardSwish

- **描述**：激活函数
- **规格约束**：无限制
    - 说明:ONNX扩展算子

InstanceNormalization

- **描述**：单例归一化
- **规格约束**：无限制

---

exLayerNorm

- **描述**：层归一化
- **规格约束**：无限制
    - 说明:ONNX扩展算子

Less

- **描述**：小于
- **规格约束**：无限制

LessOrEqual

- **描述**：小等于
- **规格约束**：无限制

LogSoftmax

- **描述**：激活函数
- **规格约束**：batchsize仅支持1

LpNormalization

- **描述**：Lp归一化
- **规格约束**：无限制

---

exLRN

- **描述**：局部响应归一化
- **规格约束**：无限制
    - 说明:ONNX扩展算子

MatMul

- **描述**：多维矩阵相乘
- **规格约束**：无限制(支持四维x四维、四维x三维计算)

Max

- **描述**：取最大值
- **规格约束**：无限制

MaxPool

- **描述**：最大池化
- **规格约束**：无限制

MaxRoiPool

- **描述**：区域最大池化
- **规格约束**：无限制

MaxUnpool

- **描述**：反向最大池化
- **规格约束**：无限制

---

exMish

- **描述**：激活函数
- **规格约束**：无限制
    - 说明:ONNX扩展算子

Min

- **描述**：取最小值
- **规格约束**：无限制

Mul

- **描述**：乘法
- **规格约束**：无限制

Pad

- **描述**：填充
- **规格约束**：无限制

Pow

- **描述**：指数计算
- **规格约束**：无限制

---

exProposal

- **描述**：区域提议网络
- **规格约束**：batchsize仅支持1
    - 说明:ONNX扩展算子

ReduceMax

- **描述**：沿指定维度计算Max
- **规格约束**：输出维度不能超过4维

ReduceMean

- **描述**：沿指定维度计算Mean
- **规格约束**：输出维度不能超过4维

ReduceSum

- **描述**：沿指定维度计算Sum
- **规格约束**：输出维度不能超过4维

ReduceMin

- **描述**：沿指定维度计算Min
- **规格约束**：输出维度不能超过4维

Reorg

- **描述**：数据重排
- **规格约束**：无限制

Reshape

- **描述**：数据形状改变
- **规格约束**：无限制

Resize

- **描述**：数据宽高方向缩放
- **规格约束**：支持插值方式bilinear;nearest2d

ReverseSequence

- **描述**：序列翻转
- **规格约束**：无限制

---

exRMSNorm

- **描述**：均方根归一化
- **规格约束**：无限制
    - 说明:ONNX扩展算子

RoiAlign

- **描述**：区域对齐池化
- **规格约束**：仅支持AvgPoolMode,batchsize仅支持1

ScatterND

- **描述**：N维索引取数
- **规格约束**：无限制

Sin

- **描述**：正弦函数
- **规格约束**：无限制

Slice

- **描述**：切片操作
- **规格约束**：batchsize仅支持1

Softmax

- **描述**：激活函数
- **规格约束**：batchsize仅支持1
    - 说明:与ONNXOPSET11规范一致

---

exSoftmax13

- **描述**：激活函数
- **规格约束**：batchsize仅支持1
    - 说明:ONNX扩展算子，与ONNXOPSET13规范一致

SpaceToDetph

- **描述**：空间方向向通道方向转换
- **规格约束**：无限制

Split

- **描述**：拆分数据
- **规格约束**：无限制

Sqrt

- **描述**：求平方根
- **规格约束**：无限制

Squeeze

- **描述**：压缩数据维度
- **规格约束**：无限制

Sub

- **描述**：减法
- **规格约束**：无限制

Tanh

- **描述**：双曲正切函数
- **规格约束**：无限制

Tile

- **描述**：扩充拷贝数据
- **规格约束**：batchsize仅支持1,不支持broadcasting

Transpose

- **描述**：转置计算
- **规格约束**：无限制

Upsample

- **描述**：上采样
- **规格约束**：支持插值方式bilinear;nearest2d

Not

- **描述**：按元素取非
- **规格约束**：无限制

---

where

- **描述**：通过mask取数
- **规格约束**：无限制

Erf

- **描述**：误差函数
- **规格约束**：无限制

Floor

- **描述**：向下取整函数
- **规格约束**：无限制

Mod

- **描述**：取模函数
- **规格约束**：无限制

---

exMeanVarianceNormalization

- **描述**：均方差归一化函数
- **规格约束**：无限制

And

- **描述**：与函数

- **规格约束**：无限制

GatherElements

- **描述**：元素收集函数
- **规格约束**：无限制

Log

- **描述**：Log函数
- **规格约束**：无限制

---

exDataConvert

- **描述**：精度转换
- **规格约束**：无限制

SpaceToDepth

- **描述**：维度变换
- **规格约束**：无限制

SoftmaxMask

- **描述**：where融合softmax的激活函数
- **规格约束**：无限制

TransposeReshape

- **描述**：组合OP
- **规格约束**：无限制

ReshapeTranspose

- **描述**：组合OP
- **规格约束**：无限制

ReduceL2

- **描述**：沿某一轴的L2范数
- **规格约束**：无限制

LayerNormalization

- **描述**：层标准化
- **规格约束**：无限制

PassThrough

- **描述**：直接连接
- **规格约束**：无限制

Expand

- **描述**：张量数据复制或扩展
- **规格约束**：无限制

Sigmoid

- **描述**：激活函数
- **规格约束**：无限制

Tanh

- **描述**：三角函数
- **规格约束**：无限制

Cos

- **描述**：三角函数
- **规格约束**：无限制

Sin

- **描述**：三角函数
- **规格约束**：无限制

---

exSwish

- **描述**：激活函数
- **规格约束**：无限制

---

exGlu

- **描述**：激活函数
- **规格约束**：无限制

---

exGelu

- **描述**：激活函数
- **规格约束**：无限制

---

exWindow

- **描述**：exWindow来源于Swin-Transformer的window_partition和window_reverse操作
- **规格约束**：无限制

---

exNorm

- **描述**：当mode设置为‘pixel’时，exNorm将使用PixelNorm进行归一化。mode缺省时，exNorm 默认使用LayerNorm进行归一化。
- **规格约束**：无限制

Round

- **描述**：数据饱和
- **规格约束**：无限制

TopK

- **描述**：获取数值排行前K个数据
- **规格约束**：无限制

---

exSwooshR

- **描述**：激活函数
- **规格约束**：无限制

---

exSwooshL

- **描述**：激活函数
- **规格约束**：无限制

OneHot

- **描述**：独热编码
- **规格约束**：无限制

Cumsum

- **描述**：指定维度上进行累加
- **规格约束**：无限制

GPU OPs 规格介绍

注:

- RV1106/RV1103/RV1106B/RV1103B/RV1126B/RK2118不支持GPU 算子
- RK356X暂未支持GridSample,exGelu

GridSample

- **描述**：插值采样
- **规格约束**：仅支持FP16,插值算法仅支持bilinear,padding_mode仅支持zeros

GRU

- **描述**：门控循环单元
- **规格约束**：仅支持hidden_size是4对齐的情况,不支持input_size和hidden_size小于4的情况

---

exGelu

- **描述**：激活函数
- **规格约束**：暂支持Int8实现

模型输入输出限制说明

模型输入限制说明

1. 该对齐约束仅针对零拷贝 API，普通 API 无此对齐约束
2. 输入宽的对齐要求可从零拷贝 API 中的 w_stride 属性查询到，注意: w_stride 不支持更改
3. 仅对输入宽(width)在不同的通道(channel)条件下有对齐要求，其他无约束
4. 若输入不需要 mean 和 scale，需要将 mean 和 scale 配置为 0 和 1
5. 若通道(channel)>4，则 mean/scale 将统一使用第一个数值，即 mean[0]和 scale[0]
6. 若首层为浮点类型则没有 quant 操作
7. RV1106/RV1103/RV1103B 不支持 CPU 的 mean/scale/quant 操作
8. 详细的用法请参考《Rockchip_RKNPU_User_Guide_RKNN_SDK》

RK3566/RK3568

<table>
<tr><th rowspan="2">芯片
平台</th><th rowspan="2">模型首层
精度类型</th><th rowspan="2">输入
维度</th><th rowspan="2">首层设置输
入数据类型</th><th rowspan="2">mean/scale/quant
后端实现设备</th><th colspan="2">输入宽（width）对齐要求单位：元素个数</th><th colspan="2">输入宽（width）大小限制</th></tr>
<tr><th>当输入通道（channel）为 1,3,4</th><th>当输入通道（channel）非 1,3,4</th><th>当输入通道（channel）为 1,3,4</th><th>当输入通道（channel）非 1,3,4</th></tr>
<tr><td rowspan="8">RK3566
/3568</td><td rowspan="4">int8</td><td rowspan="8">4 维度</td><td>uint8</td><td rowspan="2">NPU</td><td rowspan="4">8</td><td rowspan="4">1</td><td rowspan="4">各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式：
1. width * dilation_kernel_h < 1024*N
2. 2. width <= 4096
其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下：
Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 128)
Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 128)</td><td rowspan="4">无限制</td></tr>
<tr><td>int8</td></tr>
<tr><td>float16</td><td rowspan="2">CPU</td></tr>
<tr><td>其他类型
输入对齐要
求可能变动</td></tr>
<tr><td rowspan="4">float16</td><td>uint8</td><td rowspan="4">CPU</td><td rowspan="4">4</td><td rowspan="4">1</td><td rowspan="4">各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式：
1. width * dilation_kernel_h < 1024*N
2. 2. width <= 4096
其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下：
Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 128)
Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 128)</td><td rowspan="4">无限制</td></tr>
<tr><td>int8</td></tr>
<tr><td>float16</td></tr>
<tr><td>其他类型
输入对齐要
求可能变动</td></tr>
<tr><td>无限制</td><td>非 4 维</td><td>非限制</td><td>CPU</td><td>1</td><td>1</td><td>无限制</td><td>无限制</td></tr>
</table>

RK3588

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4
RK3588	int8	4 维度	uint8	NPU	16	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 8192 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 3) Depthwise Convolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 2048), 3) ConvTranspose/Deconvolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 3)	无限制
			int8					
			float16	CPU				
			其他类型输入对齐要求可能变动					
	float16		uint8	CPU	8	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 1024 * N 2. width <= 8192 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 3) Depthwise Convolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 2048), 3) ConvTranspose/Deconvolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 3)	无限制
			int8					
			float16					
			其他类型输入对齐要求可能变动					
	无限制	非 4 维	非限制	CPU	1	1	无限制	无限制

RV1103/RV1106

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4（声明见注释 9）	当输入通道（channel）非 1,3,4
RV1103/1106	int8	4 维度	uint8	NPU	16	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制

RK3562

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4
RK3562	int8	4 维度	uint8	NPU	16	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
			int8					
			float16	CPU				
			其他类型输入对齐要求可能变动					
	float16		uint8	CPU	8	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
			int8					
			float16					
			其他类型输入对齐要求可能变动					
	无限制	非 4 维	非限制	CPU	1	1	无限制	无限制

RK3576/RV1126B

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4
RK3576 / RV1126B	int8	4 维度	uint8	NPU	16	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 2048) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
			int8					
			float16	CPU				
			其他类型输入对齐要求可能变动					
	float16		uint8	CPU	8	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 2048), 3) ConvTranspose/Deconvolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
			int8					
			float16					
			其他类型输入对齐要求可能变动					
	无限制	非 4 维	非限制	CPU	1	1	无限制	无限制

RK2118

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4
RK2118	float16	4 维度	float16	CPU	4	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 3. width * dilation_kernel_h <= 2048 * N 4. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
		非 4 维			1	1		

RV1103B

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4（声明见注释 9）	当输入通道（channel）非 1,3,4
RV1103B/RV1106B	int8	4 维度	uint8	NPU	8	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
			int8					

模型输出限制说明

1. 如果输出 tensor 类型是 NHWC 的，输出转换是 NPU 实现的输出，则有对齐要求，CPU 实现的没有对齐要求。
2. 输出精度类型 int8/float16 表示模型最后一层原始输出的数据类型。
3. NCHW 输出，如果是 NPU 实现采用零拷贝接口则输出内存开辟的 size 以 query 出来的 size 为准。
4. NC1HWC2 输出，输出内存开辟的 size 以 query 出来的 size 为准。

RK3566/RK3568

芯片平台	模型输出精度类型	输出维度	设置输出 Layout	Channel 对齐要求	H*W 对齐要求
RK3566/RK3568	int8	4 维度	NCHW	无	无
			NHWC	8 对齐	无
			NC1HWC2	最后一层卷积类算子，16 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	float16		NCHW	无	无
			NHWC	4 对齐	
			NC1HWC2	最后一层卷积类算子，8 对齐，最后一层非卷积类算子 4 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	无限制	非 4 维度	UNDEFINE	无	无

RK3588

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RK3588	int8	4 维度	NCHW	无	无
			NHWC	16 对齐	无
			NC1HWC2	最后一层卷积类算子，32 对齐，最后一层非卷积类算子 16 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	float16		NCHW	无	无
			NHWC	8 对齐	无
			NC1HWC2	最后一层卷积类算子，16 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	无限制	非 4 维度	UNDEFINE	无	无

RV1103/RV1106

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RV1103 /RV1106	int8	4 维度	NC1HWC2	最后一层卷积类算子，32 对齐，最后一层非卷积类算子 16 对齐	H*W 要 4 对齐
			NHWC	无	无

RK3562

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RK3562	int8	4 维度	NCHW	无	无
			NHWC	无	无
			NC1HWC2	最后一层卷积类算子，32 对齐，最后一层非卷积类算子 16 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	float16		NCHW	无	无
			NHWC	无	无
			NC1HWC2	最后一层卷积类算子，16 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	无限制	非 4 维度	UNDEFINE	无	无

RK3576/RV1126B

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RK3576/ RV1126B	int8	4 维度	NCHW	无	无
			NHWC	无	无
			NC1HWC2	最后一层卷积类算子，32 对齐，最后一层非卷积类算子 16 对齐	H*W 要 4 对齐
			UNDE-FINE	无	无
	float16		NCHW	无	无
			NHWC	无	无
			NC1HWC2	最后一层卷积类算子，16 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			UNDE-FINE	无	无
	无限制	非 4 维度	UNDE-FINE	无	无

RK2118

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RK2118	float16	4 维度	NC1HWC2	最后一层卷积类算子，8 对齐，最后一层非卷积类算子 4 对齐	H*W 要 4 对齐
			NHWC	无	无
		非 4 维度	UNDEFINE	无	无

RV1103B/RV1106B

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RV1103B/RV1106B	int8	4 维度	NC1HWC2	最后一层卷积类算子，8 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			NHWC	无	无

特殊说明

dilated_kernel_h

`dilated_kernel_h = kernel_h * dilations_h - dilations_h + 1`

dilated_kernel_w

`dilated_kernel_w = kernel_w * dilations_w - dilations_w + 1`